

台灣自編大學教科書

@prof

機器人學導論

機構、感知、控制與具身人工智慧

第 六 篇

系統整合、產業應用與專題

第 34 章 系統整合與產業應用

第 35 章 法規、商業化與社會接受度

第 36 章 總整專題方法論

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

2026 年 9 月

第六篇 篇章導讀

前五篇教你**機器人學是什麼**，這一篇要教你**如何用它做出真正能運作的東西**。

這一篇的性質與前面五篇截然不同。前三十三章的內容是「知識」——運動學、動力學、控制、感知、規劃、學習，每一項都有明確的理論與可驗證的答案。但這三章談的是「判斷」：什麼題目值得做、什麼需求是真的、什麼風險該先驗證、什麼時候該用機構取代演算法、以及——什麼時候該誠實地說「這件事我做不到」。

第 34 章（系統整合與產業應用）從一個殘酷的統計數字開始：機器人專案的失敗原因中，技術只佔一成。需求不明佔 35%、環境假設錯誤佔 25%。一個在實驗室成功率 95% 的系統，到工廠第一天就停了七次——而那七次的原因，沒有一個是機器人學的問題。這一章教的，是把技術變成一個能在產線上跑三年的系統所需要的方法論。

第 35 章（法規、商業化與社會接受度）處理另一道門檻：當機器人走出工廠、走進有人的地方。餐廳老闆的一個問題讓整個團隊愣住——「如果它燙傷客人，是誰要賠？」這個問題的答案不在程式裡，而在法律、保險與社會共識裡。這一章談認證、責任、隱私、商業模式，以及一個常被忽略的事實：使用者信任的不是最聰明的機器人，而是最可預測的機器人。

第 36 章（總整專題方法論）是全書的最後一章，也是所有內容的最終整合。它從兩組學生的對照開始：一組做了五種功能但每種都不能用，另一組只做一件事卻測了 1,260 次並分析了每一次失敗。這一章教的是如何選題、如何界定範圍、如何規劃、如何測試、如何從失敗中提取知識、以及如何讓別人理解你的價值。

這一篇要回答的六個問題

- 實驗室 95% 的系統，為什麼在工廠第一天就停了七次？
- 客戶說「我要一台機器手臂」，他真正需要的是什麼？
- 當誤差無法降低時，還有什麼辦法？
- 如果它燙傷客人，是誰要賠？
- 做出原型之後，距離「可以上市」還有多遠？
- 一個專題的價值，究竟從哪裡來？

三條貫穿全篇的線索

線索一：先讓環境變得可控，再讓演算法變得聰明。這是第 34 章最核心的原則，也是第 33 章形態計算在產線層級的延伸。一個定位治具能讓視覺定位完全省略；一個成本兩萬的倒角，勝過花兩百萬提升精度。在把問題丟給演算法之前，先問能不能把問題本身變簡單。

線索二：盡早失敗，而且要失敗得便宜。第 34 章的 POC、第 35 章的場域試驗、第 36 章的最小可行系統——三者是同一個思維：**與其把系統做完美再驗證，不如盡早用最粗糙的版本暴露最大的風險。**因為在第二階段發現不可行，損失是三週；拖到現場驗收才發現，損失是九個月與客戶的信任。

線索三：證明比嘗試更有價值。這是第 36 章的核心，也貫穿了整個第六篇。一個能明確說出「在什麼條件下、測了幾次、成功率多少、失敗的原因是什麼」的系統，遠勝於一個宣稱「功能很多」卻無法量化的系統。而一個被充分理解的失敗，比一個沒被理解的成功更有價值。

讀完這一篇之後

你將能夠：運用系統工程的方法把技術變成可運作的系統、以五問法釐清真實需求、用誤差預算與經濟評估做技術決策、規劃產品的認證與上市路徑、設計符合社會期待的人機互動、以及——**獨立完成一個有價值的專題並讓別人理解它。**

但更重要的是三種判斷力：知道何時該用哪個工具、知道每個方法的邊界、以及知道問題的本質。因為很多時候，正確的答案不是更好的演算法，而是重新定義問題。

第 36 章的最後留下了一句話，也是這本書的最後一句話：**它會動，而那是你讓它動的。現在，換你了。**

本篇章節配置

章次	章名	核心內容	叫獸開講
第 34 章	系統整合與產業應用	V 模型、需求釐清、介面與治具、誤差預算、OEE、投資評估、安全法規	「實驗室 95%，為什麼工廠第一天就停了七次？」——技術只佔三成
第 35 章	法規、商業化與社會接受度	認證體系、產品責任、個資與隱私、RaaS、TRL 與鴻溝、人機互動設計	「如果它燙傷客人，是誰要賠？」——技術可行不等於可以上市
第 36 章	總整專題方法論	選題、範圍界定、倒推規劃、最小可行系統、量化測試、失敗分析、發表	「做到一件事的 95%，勝過五件事的 60%」——證明比嘗試更有價值

第六篇 系統整合、產業應用與專題

第 34 章

系統整合與產業應用

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩格對照：上格是明亮整潔的實驗室，機械手臂穩定地抓取乾淨的樣品，旁註「實驗室成功率 95%」；下格是真實的工廠現場——黃色照明、輸送帶持續移動、零件沾著切削油，旁註「現場第一天停機七次」，並以五個小標籤標出失敗原因（輸送帶不停、燈色不同、班別光線差異、油污、震動）。右半為 V 模型的圖示：左臂由上而下標「使用者需求 → 系統規格 → 子系統規格 → 元件設計」，右臂由下而上標「元件測試 → 子系統測試 → 系統測試 → 現場驗收」，兩臂之間以虛線橫向連接對應的層級。下緣文字：技術只佔三成，另外七成是把問題想清楚。

叫獸開講 「幫我把桌上的杯子拿到廚房」——這句話難在哪裡？

一家做汽車零件的工廠找上叫獸，希望導入機器人取代人工的檢測與包裝。他們的需求聽起來很單純：「一台手臂，把零件從輸送帶拿起來，檢查有沒有瑕疵，好的放 A 箱、壞的放 B 箱。」

小翰很有信心：「這不就是第 21 章的抓取加第 22 章的視覺辨識嗎？我一個學期就能做出來。」

三個月後，小翰帶著做好的系統去現場測試。實驗室裡成功率 95% 的系統，在工廠裡第一天就停了七次。

叫獸帶著全班去看。「說說看，發生了什麼事。」

小翰翻著筆記：「第一，輸送帶不會停——我以為零件會靜止，結果它一直在動。第二，工廠的燈是黃光，我在實驗室用白光校正的相機，顏色全跑掉了。第三，早班和晚班的光線不一樣。第四，零件上有切削油，跟我測試用的乾淨樣品完全不同。第五……」

「停。」叫獸打斷他，「這五個問題，有哪一個是**機器人學**的問題？」

小翰愣住了。小妤說：「好像……都不是。都是環境的問題。」

「這就是今天的第一課。」叫獸說，「一個**機器人系統**的失敗，通常不是因為演算法不夠好，而是因為對環境的假設錯了。你們前面三十三章學的東西，在真實的工廠裡只佔工作量的三成。」

他在白板上寫了一個數字：「另外七成是什麼？釐清需求、界定範圍、設計治具、規劃動線、訂定驗收標準、教育操作人員、處理異常、以及回答『這台機器壞了誰來修』。」

小翰不服氣：「那前面學的不就沒用了？」

「恰恰相反。」叫獸搖頭，「正因為你懂那三成，你才知道哪些需求做得到、哪些做不到、以及該用什麼治具把不可能變成可能。第 20 章那個倒角記得嗎？一個成本幾乎為零的機構，把容差放寬了 41 倍。這種判斷，只有懂技術的人做得出來。」

最後他補了一句：「今天要教的，是如何把知識變成一個真的能在工廠裡跑三年的系統。這需要的不只是技術，還有一種思維方式——**系統工程**。」

本章為什麼重要

前面三十三章教的是**技術**。但技術本身不會自動變成一個能用的系統——這中間隔著一個常被低估的鴻溝，稱為**系統整合**。

這個鴻溝的具體表現是：

- **實驗室 95%，現場 60%**：環境的差異遠比想像中大（這正是第 30 章 Sim-to-Real 落差的產業版本）。
- **技術可行 ≠ 商業可行**：一個技術上完美但投資回收要八年的方案，不會有人買單。
- **做得出來 ≠ 用得下去**：若操作員不會用、維修人員不敢修、主管看不懂報表，系統就會被閒置。

本章要建立的是一套「把技術變成系統」的方法論。它涵蓋：如何釐清真正的需求（而非客戶說出口的需求）、如何界定範圍以避免失控、如何設計整合的架構、如何評估經濟效益、如何規劃安全與驗收、以及如何處理部署後的維運。

這些內容在多數機器人教科書中缺席，卻是**業界最看重的能力**。一個能寫出 SLAM 演算法的工程師很有價值；但一個能判斷「這個案子該不該接、該怎麼報價、風險在哪裡」的工程師，**價值更高且更稀缺**。

學習目標

- 能說明系統工程的 V 模型並運用於機器人專案。
- 能區分表面需求與真實需求，並運用五問法釐清。
- 能撰寫可驗證的需求規格並定義驗收標準。
- 能說明範圍潛變的成因與三種控制方法。
- 能設計機器人工作站的整合架構並辨識關鍵介面。
- 能說明治具與周邊設計如何降低技術難度。
- 能計算投資回收期並評估專案的經濟可行性。
- 能運用風險評估方法辨識並降低專案風險。
- 能說明機器人安全標準的體系與風險評估流程。
- 能規劃系統的驗收測試與交付流程。
- 能說明部署後的維運需求與常見的失效模式。

- 能為一個真實的產業需求提出完整的系統提案。

先備知識

本章需要的是**前面三十三章的整體理解**，而非特定的章節。特別會用到：第 7 章的安全標準、第 21 章的抓取、第 22 章的視覺、第 29 章的軟體架構、第 30 章的 Sim-to-Real 落差、第 33 章的倫理與社會考量。

本章沒有數學推導，但有大量的**工程判斷**。建議讀法：把每一節的內容對照自己做過的專題，思考「當時若知道這些，會怎麼做得不一樣」。

核心名詞中英對照

表 34-1 本章核心名詞

中文	英文	說明
系統整合	System Integration	把各子系統組合為可運作整體的工程
系統工程	Systems Engineering	從需求到交付的整體工程方法論
V 模型	V-Model	需求分解與驗證對應的開發流程
需求規格	Requirements Specification	明確定義系統應達成什麼
範圍潛變	Scope Creep	專案範圍不受控地擴大
工作單元	Work Cell	機器人與周邊構成的作業單位
治具	Fixture / Jig	固定或定位工件的裝置
節拍時間	Cycle Time	完成一個作業循環所需的時間
稼動率	Availability / Uptime	設備實際可運作時間的比例
投資回收期	Payback Period	累積效益抵過投資所需的時間
整體設備效率	OEE	稼動率×性能×良率的綜合指標
風險評估	Risk Assessment	辨識危害並評估其嚴重度與機率
安全等級	Performance Level, PL	安全功能可靠度的分級
驗收測試	Acceptance Test	確認系統符合規格的正式測試
現場驗收	Site Acceptance Test, SAT	在客戶現場進行的驗收
平均故障間隔	MTBF	兩次故障之間的平均時間
平均修復時間	MTTR	從故障到恢復的平均時間
預防保養	Preventive Maintenance	定期執行以避免故障的保養

34.1 系統工程的思維

34.1.1 為何需要方法論

一個機器人專案的失敗，很少是因為「某個演算法不夠好」。統計上，失敗的原因依序是：

表 34-2 機器人專案失敗的主要原因

原因	佔比 (概略)	說明
需求不明確或改變	~35%	客戶說不清楚要什麼，或做到一半改需求
對環境的假設錯誤	~25%	現場的光線、震動、髒污與實驗室不同
整合的介面問題	~20%	各子系統單獨都對，接起來就出錯
技術本身的困難	~10%	真正的演算法或硬體限制
人的因素	~10%	操作員不會用、維修人員不支持

注意「技術本身」只佔一成。這不是說技術不重要——而是說其他九成若沒處理好，技術再好也沒用。

系統工程 (Systems Engineering) 就是處理這九成的方法論。

34.1.2 V 模型

系統工程最經典的框架是 **V 模型**。它的形狀說明了一切：左邊是逐層分解、右邊是逐層驗證，且每一層的分解都對應一層的驗證。

左側 (分解)：使用者需求 → 系統規格 → 子系統規格 → 元件設計

右側 (驗證)：元件測試 → 子系統測試 → 系統測試 → 使用者驗收

每一層的規格，都必須在對應的層級被驗證。

表 34-3 V 模型在機器人專案中的對應

階段	左側：定義什麼	右側：驗證什麼	本書相關章節
使用者需求	「我要自動化檢測與包裝」	現場驗收：實際生產三天	34.2 節
系統規格	節拍 8 秒、良率 99.5%、稼動率 95%	系統測試：連續運行測試	34.2、34.9 節
子系統規格	視覺辨識率 99.8%、抓取成功率 99.9%	子系統測試：各自的指標測試	第 21、22 章
元件設計	相機型號、夾爪設計、控制器程式	元件測試：單元測試	第 5、6、29 章

V 模型最重要的洞見是「驗證的層級對應」。一個常見的錯誤是：只做元件測試（「我的視覺辨識率 99.8%」），卻沒做系統測試（「整條線的良率是多少」）。**兩者可能天差地別——因為誤差會在系統層級累積（34.5 節）。**

34.1.3 迭代與 V 模型的結合

V 模型看起來是瀑布式的（一路往下再一路往上），這在軟體界已被批評多年。但在機器人專案中，它仍然有價值，只是需要迭代地執行：

大 V：整體專案的一次完整流程（數月）

小 v：每個子系統的快速迭代（數天到數週）

用小 v 快速試錯，用大 V 確保整體不偏離。

具體的作法是：先用第 30 章的模擬快速跑幾輪小 v（驗證概念可行）、再進入實體的原型（驗證關鍵風險）、最後才是完整的大 V（正式開發與驗收）。這正是 34.4 節「先做概念驗證」的理由。

34.2 需求：最關鍵也最容易出錯的一步

34.2.1 客戶說的不是他要的

開頭故事中，客戶的需求是「把零件拿起來、檢查、分類」。這聽起來很明確——但它其實幾乎什麼都沒說。

沒說清楚的至少有十件事：

- 零件的**種類有幾種**？未來會不會增加？
- 輸送帶會不會停？**節拍時間**是多少？
- 「瑕疵」的定義是什麼？**多小的刮痕算瑕疵**？
- 誤判的代價：把好的判成壞的（浪費）vs 把壞的判成好的（客訴）——**哪個嚴重**？
- 要求的**良率與稼動率**是多少？
- 現場的**空間**有多大？電力、氣源、網路呢？
- **誰來操作**？他們的技術水準如何？
- **壞了誰來修**？有沒有備品？
- **預算與投資回收期**的要求？

- 未來的擴充：這條線會不會改產品？

這十個問題若沒問清楚，專案幾乎必定出問題。而客戶通常不會主動說——因為他們不知道這些會影響什麼。

34.2.2 五問法：挖到真實的需求

一個實用的技巧是反覆追問「為什麼」，直到找出真正的目的。

◎ 五問法的實例

客戶說：「我要一台機器手臂。」問一：為什麼需要手臂？——「因為要把零件搬到檢測站。」問二：為什麼要搬到檢測站？——「因為檢測設備在那裡。」問三：為什麼檢測設備不能移到產線上？——「呃……好像可以，只是從來沒想過。」問四：那真正的目的是什麼？——「減少人力、提高檢測的一致性。」問五：現在的人力成本與誤判率是多少？——「兩班共四人，誤判率約3%。」結論：真正的需求是「降低檢測的人力與誤判率」，而不是「一台機器手臂」。也許把檢測設備移到線上、加裝一套視覺系統，成本只有手臂方案的三分之一。問對問題，往往比解對問題更有價值。

這個技巧的價值在於：它能找出「更便宜的解法」。而在產業中，便宜且夠用的方案，永遠勝過昂貴而完美的方案。

34.2.3 可驗證的需求規格

需求必須寫成可驗證的形式。比較兩種寫法：

表 34-4 需求的兩種寫法

模糊的寫法（不可驗證）	明確的寫法（可驗證）
「系統要快」	節拍時間 ≤ 8.0 秒/件（連續 100 件的平均）
「辨識要準」	瑕疵漏檢率 $\leq 0.1\%$ 、誤檢率 $\leq 2\%$ （以 500 件標準樣本測試）
「要穩定」	稼動率 $\geq 95\%$ （連續 30 天，扣除計畫性保養）
「要好操作」	操作員經 2 小時訓練後能獨立完成換線與異常排除
「要安全」	符合 ISO 10218 與 ISO/TS 15066，經風險評估達 PL d

右欄的每一條都能用一個明確的測試來判定通過與否。這不只是文書工作——它是驗收時避免爭議的唯一保障。

◎ 一個慘痛的教訓

一個真實的案例：某團隊交付了一套視覺檢測系統，合約寫「辨識準確率 99%」。驗收時客戶用自己現場的樣本測試，準確率只有 92%。爭議點在於：團隊用的是乾淨的標準樣本，客戶用的是沾了油、有輕微變形的實際件。合約沒有定義「用什麼樣本測試」。最終雙方各退一步，團隊免費追加了三個月的工作。教訓是：需求規格中必須明確定義「測試的條件與樣本」——這比數字本身更重要。

34.2.4 範圍潛變

範圍潛變 (Scope Creep) 是專案殺手：需求在過程中不斷增加，但時程與預算沒有跟著調整。

它的成因通常是善意的：客戶看到成果後想到更多可能、工程師想把事情做得更完美、業務為了成交而承諾額外的功能。

三種控制方法：

- **明確的範圍文件**：白紙黑字寫下「本專案包含什麼、不包含什麼」。後者尤其重要——明確寫出「不含：多品項切換、不含產線連動、不含遠端監控」。
- **變更管理流程**：任何新增的需求都要重新評估時程與費用，並經雙方書面同意。這不是刁難，而是保護雙方。
- **分階段交付**：把「一定要有」與「有更好」分開，先交付核心功能。這也降低了雙方的風險。

34.3 整合的架構

34.3.1 工作單元的組成

一個機器人工作站 (work cell) 遠不只是「一台機器人」。以開頭故事的檢測包裝站為例，完整的組成包含：

表 34-5 典型機器人工作單元的組成

類別	元件	常被低估之處
機器人本體	手臂、控制器、示教器	—
末端工具	夾爪、快換裝置、氣壓管路	線材與氣管的走線常造成干涉
感測	相機、光源、力覺、觸發訊號	光源比相機更關鍵 (34.3.3 節)
治具與周邊	定位治具、輸送帶、料倉、擋停器	這部分常佔一半以上的工時

類別	元件	常被低估之處
安全裝置	圍籬、光柵、安全門、急停	必須通過風險評估 (34.7 節)
控制與整合	PLC、工業電腦、網路、HMI	與既有產線的訊號整合最易出錯
公用設施	電力、氣源、網路、照明	現場常不足，需另行拉線

第四列與第六列是新手最容易低估的。在實務上，「機器人本體與程式」往往只佔專案工時的 30%，而治具設計與訊號整合各佔 25% 以上。

34.3.2 介面：最容易出問題的地方

34.1.1 節指出「整合的介面問題」佔失敗原因的 20%。原因是：各子系統單獨測試都正常，但接起來就出錯。

四類常見的介面問題：

- **時序 (Timing)**：訊號來得太早或太晚。例如：相機在零件還沒定位就拍照、或機器人在治具還沒夾緊就開始動作。**這是最常見的一類。**
- **資料格式**：座標系定義不同 (第 8 章)、單位不同 (公釐 vs 公尺)、旋轉表示不同 (歐拉角的順序，第 8.3 節)。
- **握手協定**：誰先動、誰等誰、失敗時如何回報。**若沒有明確定義，就會出現死鎖** (兩邊都在等對方)。
- **異常處理**：其中一方出錯時，另一方怎麼知道、怎麼反應。**這一項最常被完全忽略**——大家都只設計了「正常流程」。

◎ 介面規格書：一份被低估的文件

在多方合作的專案中，介面規格書 (Interface Control Document) 比任何技術文件都重要。它應該明確定義：訊號的名稱與意義、電氣規格 (電壓、極性)、時序圖 (誰在什麼時候發什麼)、資料格式與單位、握手的流程、以及**異常時的行為**。花兩天寫這份文件，能省下兩週的除錯——因為多數整合問題的根源是「雙方對介面的理解不同」，而這在文件中會立刻暴露。這也呼應了第 29 章 ROS 2 的核心價值：標準化的介面。

34.3.3 周邊設計：把難題變簡單

這是本章與前面各章最重要的連結。一個好的周邊設計，能把技術上極困難的問題變得平凡。

表 34-6 周邊設計降低技術難度的實例

困難的技術問題	周邊設計的解法	效果	本書呼應
零件位置不確定 → 需要高精度視覺	加一個定位治具	位置變成已知，視覺可省略	第 20.9.3 節的倒角
零件會滑動 → 需要力控制	加一個擋停器與夾具	位置固定，只需位置控制	第 20 章
光線變化 → 辨識不穩	加一個遮光罩與固定光源	環境變成可控的	第 22 章
零件形狀多變 → 抓取困難	加一個整列機構	姿態統一，抓取變簡單	第 21 章
節拍太緊 → 機器人太慢	改為雙工位輪流	有效節拍減半	—

這張表的每一列，都是「用機構解決演算法問題」的實例——正是第 33.3 節形態計算思維在產線層級的延伸。

一個實務的原則：

先問「能不能把環境變得可控」，再問「能不能用演算法處理不可控的環境」

前者通常便宜十倍、可靠十倍。

但也要注意其限制：治具是為特定工件設計的，換產品就要重做。因此在多品項、少量生產的場合，反而應該用彈性較高的視覺方案。這個取捨的判斷，正是系統整合工程師的核心價值。

34.4 專案的推進流程

34.4.1 七個階段

一個典型的機器人整合專案，包含七個階段：

表 34-7 機器人整合專案的七個階段

階段	主要工作	產出	典型佔比
一、需求訪談	釐清真實需求、現場勘查	需求規格書	5%
二、概念驗證 (POC)	驗證關鍵風險是否可行	可行性報告	10%
三、方案設計	架構、選型、佈局、報價	系統設計書、報價單	10%
四、詳細設計與製造	機構設計、治具製作、程式開發	圖面、程式、治具	35%
五、廠內組裝與測試	組裝、調試、內部測試 (FAT)	測試報告	20%

階段	主要工作	產出	典型佔比
六、現場安裝與驗收	運送、安裝、調機、驗收 (SAT)	驗收報告	15%
七、教育訓練與保固	操作訓練、文件交付、保固維護	手冊、訓練紀錄	5%

34.4.2 概念驗證：最划算的投資

第二階段的 POC (Proof of Concept) 是整個流程中投資報酬率最高的一步。它的目的不是做出完整的系統，而是驗證那個「若不成立則整個專案失敗」的關鍵假設。

如何找出關鍵假設？問這個問題：「這個專案最可能失敗的地方是什麼？」

- 若是「不確定視覺能不能辨識出那種瑕疵」→ POC 就做視覺測試，用現場的真實樣本。
- 若是「不確定節拍趕不趕得上」→ POC 就做一次動作的時間量測。
- 若是「不確定夾爪抓不抓得住」→ POC 就做抓取測試。

POC 的三個原則：

- 只驗證最關鍵的一兩件事，不要做成迷你版的完整系統。
- 必須用現場的真實條件——真實的樣本、真實的光線、真實的節拍。這正是開頭故事失敗的教訓。
- 時間要短（一至三週）。POC 拖太久，就失去了「快速止損」的意義。

◎ POC 的價值：早失敗比晚失敗好一百倍

假設一個專案在第二階段 (POC) 發現不可行，損失是十萬元與三週；若拖到第六階段 (現場驗收) 才發現，損失可能是三百萬元、九個月、以及客戶的信任。POC 的價值不在於「證明可行」，而在於「盡早發現不可行」。因此一個好的 POC 應該主動去測那些最可能失敗的情況，而非挑最容易的來展示。這與第 30.9 節的漸進式驗證是同一種思維——每一階段隔離不同的風險，讓失敗盡早且廉價地發生。

34.4.3 FAT 與 SAT

驗收分為兩個階段，這個區分極為重要：

FAT (Factory Acceptance Test，廠內驗收)：在整合商的工廠進行。優點是環境可控、問題容易修改。應盡可能在此階段解決所有問題。

SAT (Site Acceptance Test，現場驗收)：在客戶的實際產線進行。這裡會暴露所有「環

境假設」的問題——真實的光線、震動、髒污、以及與既有設備的互動。

經驗法則：能在 FAT 解決的問題，成本是 SAT 的十分之一

因此 FAT 應該盡可能模擬真實環境——用客戶提供的真實樣本、模擬現場的光線條件。

一個實務的建議：在 FAT 時請客戶到場參與。這有三個好處：他們能提早發現期待的落差、能參與問題的討論、且在 SAT 時不會有「這跟我想的不一樣」的爭議。

34.5 誤差預算與可靠度

34.5.1 誤差會累積

一個常見的錯誤是：各子系統單獨測試都達標，但整合後卻不達標。原因是誤差會在系統層級累積。

以「把零件插入孔中」為例，最終的位置誤差來自多個來源：

表 34-8 誤差來源的累積 (插件任務)

誤差來源	典型值	說明	本書章節
機器人重複精度	± 0.05 mm	規格書上的數字	第 9.9.2 節
機器人絕對精度	± 0.5 mm	與重複精度差十倍	第 9.9.2 節
視覺定位誤差	± 0.3 mm	含相機校正與辨識誤差	第 22.3 節
手眼校正誤差	± 0.2 mm	常被忽略	第 22.6 節
治具定位誤差	± 0.1 mm	治具本身的公差	34.3.3 節
工件本身公差	± 0.15 mm	來料的變異	—
夾持的偏移	± 0.2 mm	每次夾持的位置不完全相同	第 21 章
熱變形	± 0.1 mm	運轉數小時後的漂移	第 9 章

若把這些誤差直接相加 (最壞情況)：

$$\text{總誤差} = 0.5 + 0.3 + 0.2 + 0.1 + 0.15 + 0.2 + 0.1 = 1.55 \text{ mm}$$

遠超過多數插件任務所需的容差 (典型 $0.1 \sim 0.3$ mm)。

若假設誤差獨立且用均方根相加 (統計上較合理)：

$$\text{總誤差} = \sqrt{(0.5^2 + 0.3^2 + 0.2^2 + 0.1^2 + 0.15^2 + 0.2^2 + 0.1^2)} = 0.69 \text{ mm}$$

仍然過大。

這說明了一個重要的事實：即使每個環節都「很準」，整合後也可能完全不可行。

34.5.2 誤差預算

誤差預算 (Error Budget) 是系統設計的核心工具。它的作法是：

1. 由需求反推：任務的容差是多少？（如插件需要 $\pm 0.2\text{ mm}$ ）
2. 分配給各個環節：每個來源可以佔多少？
3. 檢查是否可行：若總和超過容差，就必須改變設計。
4. 改變設計：或提升某個環節的精度、或改變架構使某個誤差源消失。

第四步的「或」極為關鍵。以本例而言，有三種改善的方向：

表 34-9 三種改善誤差的策略

策略	作法	效果	成本
提升精度	換更精密的機器人、更好的相機	各項誤差減半	極高
消除誤差源	用治具取代視覺定位（34.3.3 節）	視覺 0.3 + 手眼 0.2 直接歸零	低
放寬容差	在孔口加倒角（第 20.9.3 節）	容差由 0.2 放寬到 1.0 mm	極低

第三列是最漂亮的解法——它不改善任何誤差，而是改變了問題本身。這正是第 20.9.3 節那個「一個倒角把容差放寬 41 倍」的實例，在系統層級的重現。

◎ 誤差預算的思維價值

做誤差預算最大的價值，不是算出那個數字，而是「知道該把資源投在哪裡」。若視覺誤差是 0.3 mm 而機器人絕對精度是 0.5 mm，那麼把相機從 500 萬畫素升級到 1200 萬畫素毫無意義——因為瓶頸不在那裡。這與第 30.4.2 節「哪些落差重要」、第 33 章「先做敏感度分析」是完全相同的思維：找出瓶頸，把資源投在那裡。

34.5.3 可靠度與稼動率

除了精度，可靠度同樣關鍵。三個核心指標：

$\text{MTBF (平均故障間隔)} = \text{總運轉時間} / \text{故障次數}$

$\text{MTTR (平均修復時間)} = \text{總修復時間} / \text{故障次數}$

$\text{稼動率} = \text{MTBF} / (\text{MTBF} + \text{MTTR})$

稼動率是客戶最在意的數字。

一個重要的觀察：提升稼動率有兩條路——增加 MTBF（更可靠）或降低 MTTR（更好修）。而後者往往便宜得多：

- 模組化設計：故障時整組更換，而非現場維修。MTTR 由數小時降到十分鐘。
- 良好的診斷：明確的錯誤訊息與指示燈，讓操作員能自行排除簡單的異常。
- 備品策略：關鍵易損件（夾爪指片、感測器、氣管）現場備有存貨。
- 遠端診斷：工程師能遠端查看日誌與狀態，不需到場。

這也是第 29.9.3 節「每次都錄 rosbag」的產業版本——沒有紀錄，遠端診斷就無從談起。

34.5.4 整體設備效率 (OEE)

製造業衡量設備表現的標準指標是 OEE：

$$\text{OEE} = \text{稼動率} \times \text{性能效率} \times \text{良率}$$

三者相乘——任何一項低落都會嚴重拉低整體。

這個相乘的結構有重要的意義。假設三項都是 90%（聽起來都不錯）：

$$\text{OEE} = 0.90 \times 0.90 \times 0.90 = 0.729 = 72.9\%$$

三個「不錯」相乘之後只有七成。

而製造業的標竿水準是 $\text{OEE} \geq 85\%$ ，這要求三項都在 95% 左右。這說明了為何工業級的要求如此嚴苛——也呼應了第 33.7.3 節「從 90% 到 99.9% 比從 0% 到 90% 更難」的觀察。

34.6 經濟評估

34.6.1 為何工程師必須懂

一個技術上完美的方案，若投資回收要八年，**不會有人買單**。因此經濟評估不是業務的事，而是工程師必須具備的判斷力——因為它會影響技術方案的選擇。

34.6.2 投資回收期

最常用也最直觀的指標是投資回收期 (Payback Period)：

$$\text{投資回收期} = \text{總投資額} / \text{年度效益}$$

製造業的常見要求是 2~3 年以內；若超過 5 年，多數企業不會投資。

總投資額包含的項目常被低估：

表 34-10 自動化專案的成本項目

項目	說明	常被低估的程度
機器人本體	手臂與控制器	低 (有明確報價)
周邊與治具	夾爪、治具、輸送帶、料倉	高 (常佔總額的 40%)
視覺與感測	相機、光源、鏡頭	中
安全設備	圍籬、光柵、安全繼電器	高 (法規要求，不可省)
系統整合工時	設計、程式、調試	極高 (常佔 30~50%)
安裝與試車	運送、安裝、現場調機	中
產線停機損失	安裝期間無法生產	極高 (常被完全忽略)
教育訓練	操作與維護人員的訓練	低

倒數第二列特別值得注意。若一條產線每天產值一百萬，安裝停機五天就是五百萬的損失——這可能比機器人本身還貴。因此實務上常採「分段導入」或「假日安裝」以降低這項成本。

34.6.3 年度效益的計算

效益不只是「省下的人力」，還包含四類：

- **人力節省**：最直接的一項。但要注意——通常不是「省下 N 個人」，而是「這 N 個人去做別的事」。若公司不裁員，效益要以「產能提升」而非「薪資節省」計算。
- **品質提升**：不良率降低所減少的報廢與客訴成本。這一項常被低估，但在高價值產品上可能是最大的效益。
- **產能提升**：機器人可以連續運轉 (不需休息、可上夜班)，使產能提升。
- **間接效益**：職業傷害減少、人力招募壓力降低、交期穩定度提升。這些難以量化但真實存在。

◎ 一個常見的誤區：只算人力

許多提案只計算「省下兩個人 × 年薪 60 萬 = 120 萬」。但這往往低估了真實的效益。以開頭故事的檢測站為例：人工檢測的誤判率 3%，若每件產品價值 500 元、月產 10 萬件，則誤判造成的損失為 $10 \text{ 萬} \times 3\% \times 500 = 150 \text{ 萬/月}$ 。若自動化能把誤判率降到 0.5%，每月省下 125 萬——遠超過人力節省的效益。因此提案時務必把品質效益納入計算，這往往是說服決策者的關鍵。

34.6.4 不只看回收期

投資回收期簡單直觀，但有三個限制：

- **忽略回收後的效益**：一台能用八年的設備，回收期兩年，但後面六年都是淨賺。
- **忽略資金的時間價值**：今天的一百萬比五年後的一百萬值錢。
- **忽略風險**：兩個回收期都是三年的方案，風險可能完全不同。

因此財務上還會用淨現值（NPV）與內部報酬率（IRR）。但對工程師而言，理解回收期已足以做技術判斷——關鍵是知道它會影響方案的選擇。

一個實用的判斷：若你的方案回收期是四年而客戶要求兩年，不要只想著降低報價——應該回頭檢視：能不能用更簡單的方案（34.2.2 節的五問法）？能不能把效益算得更完整（34.6.3 節）？能不能分階段導入以降低初期投資？

34.7 安全與法規

34.7.1 安全是法律義務

第 7 章介紹過機器人的安全概念，本節從**產業實務**的角度補充。最重要的認識是：

◎ 安全不是「做好一點」，而是法律義務

在多數國家，工業機器人的安全防護是強制性的法規要求，而非「可以斟酌的品質項目」。若因未達安全標準而發生職災，雇主與設備供應商都可能負法律責任（呼應第 33.8.3 節的責任歸屬）。因此安全的預算**不可刪減**——這是報價時必須向客戶明確說明的。

34.7.2 標準體系

機器人安全的標準是分層的，理解這個結構有助於查找正確的規範：

表 34-11 機器人安全標準的體系

層級	標準	涵蓋範圍
A 類（基礎）	ISO 12100	機械安全的通則、風險評估的方法
B 類（群組）	ISO 13849、IEC 62061	安全控制系統的設計與可靠度（PL/SIL）
	ISO 13855	安全距離的計算（光柵該裝多遠）
C 類（產品）	ISO 10218-1	機器人本體的安全要求
	ISO 10218-2	機器人系統與整合的安全要求

層級	標準	涵蓋範圍
	ISO/TS 15066	協作機器人的力與壓力限制 (第 20.10 節)

C 類標準優先於 A、B 類——若 ISO 10218 有明確規定，就依它；沒規定的部分才回到通則。

34.7.3 風險評估的流程

風險評估 (Risk Assessment) 是安全設計的核心程序，依 ISO 12100 分為五步：

1. 界定範圍：機器的使用限制、預期用途、以及可預見的誤用。
2. 辨識危害：列出所有可能的危害——夾擠、撞擊、切割、電擊、噪音、以及維修時的危害 (常被忽略)。
3. 風險估計：對每個危害評估「傷害的嚴重度 × 暴露頻率 × 迴避的可能性」。
4. 風險評價：判斷風險是否可接受。
5. 風險減低：若不可接受，依三階法採取措施 (見下)。

第五步的「三階法」有明確的優先順序，不可跳過：

第一階：本質安全設計 (消除危害本身)
第二階：工程防護 (圍籬、光柵、安全控制)
第三階：資訊警告 (標示、手冊、訓練)

必須依序考慮——不能因為「貼個警告標示比較便宜」就跳過前兩階。

第一階最常被忽略但最有效。例如：把夾點的間隙加大到手指伸不進去、把移動速度限制在安全範圍、或把危險的動作區域設計成人不會進入的位置。這正是第 7.9 節「本質安全」的體現，也與第 33.3 節的形態計算是同一種思維——用設計消除問題，而非用防護遮蔽問題。

34.7.4 協作機器人的特殊考量

協作機器人 (第 20.10 節) 常被誤解為「不需要安全防護」。這是危險的誤解。

正確的認識是：協作機器人允許在特定條件下與人共處，但仍必須經過風險評估。ISO/TS 15066 定義了四種協作模式：

表 34-12 ISO/TS 15066 的四種協作模式

模式	作法	適用
安全級監控停止	人進入時機器人停止	偶爾需要人介入的作業
手動導引	人握著機器人移動它	示教、輔助搬運
速度與間距監控	依人的距離動態調整速度	人機頻繁交錯的作業
功率與力限制	限制接觸力不致傷害	真正的並肩協作（第 20.5 節）

最後一種最常被稱為「協作」，但它的限制也最嚴格——必須經過實測驗證接觸力與壓力不超過標準值（依身體部位而異，如手部 140 N、臉部禁止接觸）。且一旦裝上尖銳的工具或抓取重物，原本的安全評估就失效了，必須重新評估。

34.8 產業應用的樣貌

34.8.1 四大應用領域

機器人的產業應用，可依「環境的結構化程度」與「任務的變異程度」分為四個象限：

表 34-13 機器人的四大應用領域

領域	環境	任務變異	代表應用	關鍵技術
傳統工業	高度結構化	低（大量重複）	焊接、噴塗、搬運、組裝	精度、速度、可靠度
彈性製造	結構化	高（多品項少量）	檢測、包裝、混線生產	視覺、快速換線
物流倉儲	半結構化	中（品項多但動作單純）	分揀、揀貨、搬運	抓取泛化、導航
服務與戶外	非結構化	高	清潔、巡檢、農業、醫療	感知、自主、安全

這四個象限的技術重點截然不同，也對應了本書不同章節的內容：

- 傳統工業依賴的是第 9~19 章的精確運動學與控制——環境已知，重點是快與準。
- 彈性製造加入了第 22 章的視覺——因為工件位置與種類會變。
- 物流倉儲需要第 21 章的泛化抓取與第 26 章的導航——品項無法窮舉。
- 服務與戶外則需要第 24~33 章的全部——不確定性、自主決策、與人共處。

34.8.2 技術成熟度的落差

一個誠實的觀察：四個領域的技術成熟度差距極大。

表 34-14 各領域的成熟度與挑戰

領域	成熟度	典型稼動率	主要挑戰
傳統工業	極成熟（數十年）	> 98%	已非技術問題，而是成本與彈性
彈性製造	成熟中	90~97%	換線時間、多品項的視覺穩定度
物流倉儲	快速發展	85~95%	抓取泛化（品項無法窮舉）
服務與戶外	early stage	70~90%	可靠度、成本、法規、社會接受度

最後一系列的「可靠度」是關鍵瓶頸。這正是第 33.7.3 節的觀察——實驗室的 90% 在真實部署中不可接受。一台每十次會失敗一次的服務機器人，會產生大量的人工介入需求，反而增加成本。

34.8.3 台灣的產業脈絡

對台灣的讀者而言，有幾個特殊的脈絡值得認識：

- **中小企業為主**：台灣製造業以中小企業居多，**多品項、少量、快速換線**是常態。這使得「大批量的專用自動化」不適用，反而需要**彈性製造**的方案。
- **缺工問題嚴重**：這使自動化的驅動力來自「**找不到人**」而非「省人力成本」——這改變了經濟評估的邏輯（34.6.3 節）。
- **產業聚落完整**：機構加工、電控、模具的供應鏈健全，**治具與周邊的製作成本相對低**。這使 34.3.3 節「用機構解決問題」的策略特別有利。
- **半導體與電子業的高精度需求**：這個領域對精度、潔淨度與稼動率的要求極高，是台灣機器人應用最成熟的場域。

一個實務的建議：台灣的機器人工程師若要創造價值，「**彈性製造**」與「**快速換線**」往往比「**追求極致精度**」更有市場——因為前者切中中小企業的痛點，而後者已有國際大廠的成熟方案。

34.9 部署與維運

34.9.1 交付不是結束

一個常見的錯誤觀念是：**驗收通過就等於專案完成**。實際上，系統要在產線上運作三到五年，維運的總成本往往超過建置成本。

交付時必須包含的四樣東西：

- **操作手冊**：不是技術文件，而是**操作員看得懂的圖文步驟**——如何開機、換線、排除常見異常。
- **維護手冊**：保養項目與週期、易損件清單與料號、故障排除的決策樹。
- **教育訓練**：且必須包含**實作**。只做簡報的訓練，一個月後就忘光了。
- **備品清單**：哪些零件應該現場備有存貨（34.5.3 節）。

34.9.2 常見的失效模式

實務上，機器人系統在部署後最常見的問題**不是程式錯誤**，而是以下六類：

表 34-15 部署後的常見失效模式

失效	成因	預防
夾爪指片磨損	長期夾持造成尺寸變化	定期更換（易損件）、監測夾持力
視覺辨識漂移	光源老化、鏡頭積灰、環境光變化	定期校正、光源亮度監測、遮光
治具鬆動或磨損	長期震動、碰撞	定期檢查與校正、加防鬆設計
氣壓不足	廠務氣源波動、管路洩漏	加裝壓力監測與警報
來料變異	供應商換料或製程變動	來料檢查、擴大系統的容忍範圍
人為變更	有人移動了相機或治具	明確的標示與固定、變更管理

第二列與第五列最值得警惕——它們是**緩慢漸進**的，不會立刻造成停機，而是使良率逐漸下降。等到有人注意到時，可能已經生產了大量的不良品。

因應之道是「**監測趨勢而非只看當下**」：記錄每天的辨識信心分數、夾持力、節拍時間，繪出**趨勢圖**。當某個指標開始漂移時就介入——這正是第 30.10 節數位學生的預測性維護思想。

34.9.3 持續改善

一個系統上線後，通常還有 10~30% 的改善空間。三個常見的著力點：

- **節拍優化**：分析動作的時間分布，找出瓶頸。常見的改善是**動作的重疊**（如相機拍照的同時機器人已開始移動）與**軌跡的平滑化**（第 15 章）。
- **參數微調**：依實際的資料調整視覺的門檻、力控制的增益、加減速的參數。
- **異常處理的完善**：上線後會遇到當初沒想到的異常。**每一次異常都應該被記錄並納入處理邏輯**。

◎ 為何「上線後才是開始」

一個系統的價值，不在於交付那一天有多完美，而在於它三年後還在穩定運轉。這需要三件事：一、可監測（能看見狀態的趨勢）；二、可維護（模組化、有備品、有文件）；三、可調整（參數外部化，第 29.5.1 節）。這三件事都必須在設計階段就考慮——若等到上線後才想加，成本會高出數倍。這也是為何本章要放在全書的最後：只有理解了完整的技術，才能在設計時做出這些前瞻的判斷。

34.10 一個完整的案例

34.10.1 情境

讓我們用開頭故事的檢測包裝站，走一遍完整的流程。

客戶的原始需求：「一台手臂，把零件從輸送帶拿起來，檢查有沒有瑕疵，好的放 A 箱、壞的放 B 箱。」

34.10.2 需求釐清（34.2 節）

經過訪談與五問法，釐清出的真實需求為：

表 34-16 釐清後的需求規格

項目	規格	驗證方式
零件種類	3 種，尺寸 30~80 mm，未來可能增至 5 種	—
節拍時間	≤ 6.0 秒/件	連續 200 件的平均
瑕疵定義	刮痕 > 0.5 mm、缺料、變形 > 1 mm	以標準樣本判定
漏檢率	$\leq 0.1\%$ （壞判成好，客訴風險）	500 件含 50 件已知不良的測試
誤檢率	$\leq 3\%$ （好判成壞，浪費）	同上
稼動率	$\geq 95\%$	連續 30 天運轉紀錄
投資回收期	≤ 2.5 年	財務試算
安全	符合 ISO 10218-2，達 PL d	風險評估報告

注意「漏檢率」與「誤檢率」的不對稱——前者要求嚴格十倍，因為客訴的代價遠高於浪費。這個判斷必須由客戶提供，工程師不能自行假設。

34.10.3 關鍵風險與 POC（34.4.2 節）

最可能失敗的地方是什麼？團隊判斷是「視覺能否在現場的光線與油污下，穩定辨識 0.5

mm 的刮痕」。

POC 的設計：

1. 向客戶索取 100 件真實樣本（含 20 件已知不良，且不清洗）。
2. 在客戶現場架設相機與光源測試（而非在實驗室）。
3. 分別在早班與晚班各測一次，觀察環境光的影響。
4. 為期兩週，產出可行性報告。

POC 的結果：在加裝遮光罩與同軸光源後，漏檢率 0.08%、誤檢率 2.1%——達標，可以進入正式開發。若沒做這一步而直接開發，很可能在 SAT 才發現不可行。

34.10.4 架構設計（34.3 節）

依 34.3.3 節「先讓環境可控」的原則，方案設計為：

表 34-17 系統架構與設計決策

問題	決策	理由
零件在輸送帶上移動	加擋停器 + 定位治具	位置變已知，視覺只需檢測不需定位
環境光變化	遮光罩 + 同軸光源	環境變可控（POC 已驗證）
3 種零件尺寸不同	可調式治具 + 自適應夾爪	換線只需調治具，不需換夾爪
節拍 6 秒偏緊	檢測與搬運平行	拍照的同時機器人移向下一件
未來可能增至 5 種	視覺模型可線上新增類別	預留擴充性（避免範圍潛變）
安全	圍籬 + 安全門開關 + 光柵	風險評估後達 PL d

注意前三列都是「用機構解決」——這正是 34.3.3 節與第 33.3 節形態計算的實踐。

34.10.5 誤差預算（34.5 節）

抓取的容差需求為 ± 1.5 mm（夾爪有 V 型槽導引）。誤差來源：

機器人絕對精度 ± 0.5 、治具定位 ± 0.3 、工件公差 ± 0.2 、夾持偏移 ± 0.2

$$\text{均方根總和} = \sqrt{(0.5^2 + 0.3^2 + 0.2^2 + 0.2^2)} = 0.66 \text{ mm}$$

$$0.66 < 1.5, \text{ 餘裕充足 } \checkmark$$

注意視覺的誤差沒有出現在這裡——因為治具已經固定了位置，視覺只負責「判斷好壞」而不負責「定位」。這正是 34.5.2 節「消除誤差源」的策略。

34.10.6 經濟評估 (34.6 節)

投資成本：

機器人與控制器 85 萬、視覺系統 45 萬、治具與周邊 60 萬
安全設備 25 萬、系統整合工時 95 萬、安裝試車 20 萬
總計 = 330 萬元

注意整合工時佔 29%、治具佔 18%——符合 34.6.2 節的觀察。

年度效益：

人力節省：2 人 × 2 班 × 60 萬 = 240 萬/年
品質效益：誤判率由 3% 降至 0.5%
月產 10 萬件 × 2.5% × 500 元 = 125 萬/月 = 1500 萬/年

品質效益遠大於人力效益 (34.6.3 節的知識框)。

保守起見，品質效益只計入四成 (因為部分不良品原本就會在下游被攔截)：

年度效益 = 240 + 1500 × 0.4 = 840 萬/年
投資回收期 = 330 / 840 = 0.39 年 ≈ 4.7 個月

遠優於 2.5 年的要求 ✓

這個案例說明了 34.6.3 節的重點：若只計算人力節省 (回收期 $330/240 = 1.4$ 年)，雖然也達標，但說服力遠不如把品質效益納入。

34.11 由整合邁向專題

34.11.1 本章的成果

本章建立了「把技術變成系統」的方法論。你現在能夠：運用 V 模型規劃專案、以五問法釐清真實需求、撰寫可驗證的規格、設計整合架構並辨識介面風險、用周邊設計降低技術難度、計算誤差預算與經濟效益、規劃安全與驗收、以及設計可維運的系統。

最重要的是三個判斷：

- 先問「能不能把環境變得可控」，再問「能不能用演算法處理不可控的環境」——前者通常便宜十倍、可靠十倍。
- 找出瓶頸，把資源投在那裡——誤差預算與敏感度分析的核心價值。
- 盡早失敗——POC 的價值不在證明可行，而在盡早發現不可行。

34.11.2 接下來

第 35 章將處理本章未涵蓋的一個面向：當機器人走出工廠、進入公共空間與家庭時，會遇到什麼技術之外的問題——法規、標準、認證、商業模式、以及社會的接受度。第 33 章已從哲學與倫理的角度談過，第 35 章則從**產業實務**的角度補完。

第 36 章則是全書的最終整合——**完整的專題方法論**。它會帶你從「選題」開始，經過規劃、開發、測試、到成果發表，並特別著重**失敗原因的分析與設計改良**。若說第 34 章教的是「產業專案怎麼做」，第 36 章教的就是「**你自己的專題怎麼做**」。

換句話說：第 34 章讓你知道業界如何把技術變成系統，第 36 章讓你親手走一遍。

完整計算例題

例題 34-1 誤差預算與方案選擇

題目：某插件任務要求容差 ± 0.25 mm。目前的誤差來源為：機器人絕對精度 ± 0.5 mm、視覺定位 ± 0.35 mm、手眼校正 ± 0.2 mm、工件公差 ± 0.15 mm、夾持偏移 ± 0.2 mm。(a) 計算最壞情況與均方根的總誤差。(b) 判斷是否可行。(c) 提出三種改善方案並比較。(d) 選擇最佳方案並計算改善後的誤差。

解 (a)：兩種計算方式：

$$\text{最壞情況 (直接相加)} = 0.5 + 0.35 + 0.2 + 0.15 + 0.2 = 1.40 \text{ mm}$$

保守的估計。

$$\begin{aligned} \text{均方根} &= \sqrt{(0.5^2 + 0.35^2 + 0.2^2 + 0.15^2 + 0.2^2)} \\ &= \sqrt{(0.25 + 0.1225 + 0.04 + 0.0225 + 0.04)} = \sqrt{0.475} = 0.689 \text{ mm} \end{aligned}$$

統計上較合理的估計。

解 (b)：即使用較樂觀的均方根計算，0.689 mm 仍是容差 0.25 mm 的 2.8 倍——完全不可行。

解 (c)：三種方案：

表 34-18 三種改善方案的比較

方案	作法	改善後誤差	成本	彈性
甲：提	換高精度機器人 (± 0.1)	$\sqrt{(0.1^2 + 0.15^2 + 0.2^2 + 0.15^2 + 0.2^2)} =$	極高	高

方案	作法	改善後誤差	成本	彈性
升精度	+ 更好的視覺 (± 0.15)	0.35 mm	(+200 萬)	
乙：消除誤差源	加定位治具，取消視覺定位	$\sqrt{(0.5^2+0.1^2+0.15^2+0.2^2)} = 0.58$ mm	低 (+15 萬)	低 (換件要改治具)
丙：放寬容差	在孔口加 1 mm 倒角 (第 20.9.3 節)	誤差不變，但容差變 1.25 mm	極低 (+2 萬)	高

分析：

- 方案甲改善到 0.35 mm，仍然超過 0.25 mm——花了兩百萬還是不行。
- 方案乙改善有限 (因為機器人的 0.5 mm 仍在)，也不行。
- 方案丙誤差不變，但容差由 0.25 放寬到 1.25 mm，而總誤差 $0.689 < 1.25$ ✓ 可行。

解 (d)：選擇方案丙 (加倒角)。改善後：

誤差 0.689 mm < 容差 1.25 mm，餘裕 45%

成本僅 2 萬元，遠低於方案甲的 200 萬。

核心的工程判讀：這個例題最重要的教訓是——當誤差無法降低時，就改變容差。方案甲與乙都在「努力降低誤差」，而方案丙改變了問題本身。這正是第 20.9.3 節、第 33.3 節形態計算的思維：在把問題丟給精度之前，先問能不能用機構讓問題消失。

但也要注意其前提：倒角必須是客戶的工件設計允許的。若工件已量產且不能修改，方案丙就不成立——此時可能要考慮在夾爪端加 V 型導引，達到同樣的效果。

例題 34-2 OEE 與稼動率的分析

題目：某產線的班別為 8 小時，其中計畫性停機 (保養、換線) 40 分鐘。實際運轉時因故障停機 25 分鐘。理論節拍 5.0 秒/件，實際共生產 4,800 件，其中不良 96 件。(a) 計算稼動率。(b) 計算性能效率。(c) 計算良率。(d) 計算 OEE 並判斷水準。(e) 若要提升 OEE 至 85%，分析應優先改善哪一項。

解 (a)：稼動率 = 實際運轉時間 / 計畫運轉時間：

計畫運轉時間 = 480 - 40 = 440 分鐘

$$\text{實際運轉時間} = 440 - 25 = 415 \text{ 分鐘}$$

$$\text{稼動率} = 415/440 = 94.3\%$$

故障停機造成 5.7% 的損失。

解 (b)：性能效率 = 理論所需時間 / 實際運轉時間：

$$\text{理論所需時間} = 4,800 \text{ 件} \times 5.0 \text{ 秒} = 24,000 \text{ 秒} = 400 \text{ 分鐘}$$

$$\text{性能效率} = 400/415 = 96.4\%$$

有 3.6% 的微停與速度損失 (如短暫的卡料、節拍略慢於理論值)。

解 (c)：良率 = 良品數 / 總生產數：

$$\text{良率} = (4,800 - 96)/4,800 = 4,704/4,800 = 98.0\%$$

不良率 2%。

解 (d)：

$$\text{OEE} = 0.943 \times 0.964 \times 0.980 = 0.891 = 89.1\%$$

超過 85% 的標竿水準 ✓

判讀：三項都在 94% 以上，相乘後 89.1%——這印證了 34.5.4 節的觀察：相乘的結構使得「每一項都要很高」。

解 (e)：目前已達 89.1%，但若要進一步提升，應分析各項的改善空間：

表 34-19 各項的改善潛力分析

項目	目前	損失	改善難度	優先度
稼動率	94.3%	5.7% (故障 25 分)	中 (需分析故障原因)	高
性能效率	96.4%	3.6% (微停與速度)	中 (需優化節拍)	中
良率	98.0%	2.0% (96 件不良)	高 (涉及製程)	低

建議優先改善稼動率，理由有三：

- 損失最大 (5.7%)。
- 故障停機通常有明確的原因，分析 25 分鐘的停機記錄，往往能找出一兩個反覆出現的問題 (如 34.9.2 節的夾爪磨損或氣壓不足)。
- 改善成本低：多數是保養與備品的問題，而非需要重新設計。

若能把故障停機由 25 分降到 10 分：

$$\text{新稼動率} = 430/440 = 97.7\%$$

$$\text{新 OEE} = 0.977 \times 0.964 \times 0.980 = 0.923 = 92.3\%$$

提升 3.2 個百分點。

例題 34-3 投資評估與方案決策

題目：某工廠考慮自動化一個包裝工位。現況為 3 班制、每班 2 人，人力成本每人每年 55 萬。三個方案的資料如下表。（a）計算各方案的投資回收期。（b）若要求回收期 ≤ 2.5 年，判斷哪些可行。（c）考慮五年的總效益，比較三者。（d）綜合判斷應選哪一個。

表 34-20 三個方案的比較

方案	投資額	取代人力	額外效益	預期壽命
甲：半自動	180 萬	3 人（每班 1 人）	無	8 年
乙：全自動（標準）	420 萬	6 人（全部）	品質效益 60 萬/年	8 年
丙：全自動（高規）	680 萬	6 人（全部）	品質效益 60 萬/年 + 產能提升 150 萬/年	10 年

解（a）：先計算各方案的年度效益：

$$\text{甲：} 3 \times 55 = 165 \text{ 萬/年}$$

$$\text{乙：} 6 \times 55 + 60 = 390 \text{ 萬/年}$$

$$\text{丙：} 6 \times 55 + 60 + 150 = 540 \text{ 萬/年}$$

年度效益。

$$\text{甲：} 180/165 = 1.09 \text{ 年}$$

$$\text{乙：} 420/390 = 1.08 \text{ 年}$$

$$\text{丙：} 680/540 = 1.26 \text{ 年}$$

投資回收期。

解（b）：三個方案的回收期都在 1.1~1.3 年，全部遠優於 2.5 年的要求 ✓

這是一個常見的情況——當人力成本高（本例 3 班 6 人，年成本 330 萬）時，多數自動化方案都能快速回收。此時回收期就不是決策的關鍵，必須看其他因素。

解 (c)：五年的累積淨效益 (效益 $\times$ 年數 - 投資)：

$$\text{甲：} 165 \times 5 - 180 = 825 - 180 = 645 \text{ 萬}$$

$$\text{乙：} 390 \times 5 - 420 = 1,950 - 420 = 1,530 \text{ 萬}$$

$$\text{丙：} 540 \times 5 - 680 = 2,700 - 680 = 2,020 \text{ 萬}$$

丙的累積效益最高。

解 (d)：綜合判斷應選方案丙，但需檢視三個前提：

- 「產能提升 150 萬」是否真的能實現？若市場需求不足，多出來的產能賣不掉，這 150 萬就不存在——此時丙的效益降為 390 萬/年，與乙相同，但投資多了 260 萬，反而較差。
- 6 個人去哪裡？若公司不裁員而是轉調，則「人力節省」應改以「產能提升」計算 (34.6.3 節)。
- 丙的技術風險是否較高？高規方案通常較複雜，稼動率可能較低。

這個例題的核心教訓：回收期只是門檻，不是決策依據。當多個方案都通過門檻時，應該比較累積效益、並檢視效益假設是否成立。而最後一點——質疑自己的假設——往往是工程師最欠缺卻最重要的能力。

實作：一個完整的系統提案

本章的實作是撰寫一份完整的系統提案。這是業界最常見的工作產出，也是把本章知識整合的最佳方式。

1. **選定題目**：選一個你熟悉的重複性工作作為自動化的對象。可以是校內的 (如實驗室的樣品分類)、家庭的 (如衣物分類)、或產業的 (若有實習經驗)。選一個你能實際觀察的。
2. **現場觀察**：實際去看那個工作至少三次，記錄：動作的步驟與時間、環境的條件 (光線、空間、干擾)、異常發生的頻率與處理方式。這一步不可省略——它是所有需求的來源。
3. **需求訪談**：訪談實際執行者與其主管。用 34.2.2 節的五問法，找出「真實的需求」而非「表面的要求」。記錄兩者的差異。
4. **撰寫需求規格**：依 34.2.3 節的原則，寫出至少 8 條可驗證的需求。每一條都要有明確的數值與測試方式。
5. **界定範圍**：明確寫出「本專案包含什麼、不包含什麼」 (34.2.4 節)。
6. **辨識關鍵風險**：這個專案最可能失敗的地方是什麼？設計一個 POC 來驗證它 (34.4.2

節)。

7. **架構設計**：繪製系統的組成圖 (34.3.1 節)，標出所有的元件與介面。**特別標出你認為最容易出問題的介面。**
8. **周邊設計**：用 34.3.3 節的思維，找出至少三個「可以用機構解決的技術問題」。
9. **誤差預算**：若涉及精度，做一次誤差預算 (34.5 節)，判斷是否可行。
10. **經濟評估**：計算投資額 (含 34.6.2 節的所有項目) 與年度效益 (含 34.6.3 節的四類)，求出投資回收期。
11. **安全評估**：依 34.7.3 節的流程做一次簡化的風險評估，列出至少五項危害與其防護措施。
12. **撰寫提案書**：整合以上內容為一份 10~15 頁的提案書，包含：現況分析、需求規格、方案架構、實施計畫、經濟效益、風險與對策。
13. **(進階) 向他人簡報**：找一位不熟悉技術的人 (如家人或其他系的同學) 聽你的提案。**若他能理解你要解決什麼問題、以及為什麼值得投資，代表你的提案是成功的。**

表 34-21 系統提案的自我檢核表

檢核項目	是否完成	備註
需求都是可驗證的 (有數值與測試方式)	☐	34.2.3 節
明確寫出了「不包含什麼」	☐	34.2.4 節
辨識出關鍵風險並設計了 POC	☐	34.4.2 節
至少找出三個「用機構解決」的機會	☐	34.3.3 節
投資額包含了整合工時與停機損失	☐	34.6.2 節
效益包含了品質與產能 (非只有人力)	☐	34.6.3 節
做了風險評估並列出防護措施	☐	34.7.3 節
考慮了維運 (備品、訓練、文件)	☐	34.9 節

國中小也看得懂：先想清楚要做什麼

假設你想做一台「自動整理書包」的機器人。你會怎麼開始？

大部分的人會直接想：「要用什麼手臂？要怎麼寫程式？」

但真正厲害的工程師會先問一堆問題：

「書包裡有幾種東西？」（課本、鉛筆盒、水壺.....如果有一百種，就很難做）

「要多快整理完？」（一分鐘還是十分鐘？差很多！）

「整理錯了會怎樣？」（把課本放錯格子沒關係，但把水壺放倒會濕掉全部）

「誰來用它？」（如果是小朋友自己用，操作就不能太複雜）

「壞掉了誰修？」（如果只有你會修，那你不在的時候怎麼辦？）

有沒有發現？這些問題都跟「怎麼寫程式」沒關係，但它們才是最重要的。

這一章要告訴你一個很重要的祕密：

在真實的世界裡做一台機器人，「技術」只佔三成，另外七成是「想清楚要做什麼」。

而且還有一個超級好用的技巧：與其把機器人做得更聰明，不如把環境變得更簡單。

舉個例子：如果書包裡的東西亂七八糟，機器人要很聰明才能認出來。但如果你在書包裡加幾個隔層，讓每樣東西都有固定的位置——**機器人就變得超簡單了！**

這就像考試一樣：與其把題目背得滾瓜爛熟，不如先看清楚題目在問什麼。**很多時候，把問題想清楚，答案就自己出現了。**

所以下次你想做什麼東西的時候，先別急著動手。**先問自己五個「為什麼」**，你會發現真正要解決的問題，常常跟你一開始想的不一樣。

叫獸提醒與常見錯誤

- 誤區一：以為專案的困難在技術。統計上技術只佔失敗原因的一成，需求不明與環境假設錯誤才是主因。
- 誤區二：接受客戶說出口的需求就開始做。應用五問法追問真實的目的——往往有更便宜的解法。
- 誤區三：需求寫成「要快」「要準」這類無法驗證的文字。必須有明確的數值、測試條件與樣本定義。
- 誤區四：沒有明確寫出「不包含什麼」。這是防止範圍潛變最有效的一句話。
- 誤區五：跳過 POC 直接開發。早失敗比晚失敗便宜一百倍，且 POC 必須用現場的真實條

件。

- 誤區六：只做元件測試而不做系統測試。誤差會在系統層級累積，各項達標不代表整體達標。
- 誤區七：先想演算法再想機構。應先問「能不能讓環境變得可控」——通常便宜十倍、可靠十倍。
- 誤區八：投資額只算機器人本體。整合工時常佔 30~50%、治具佔 40%、停機損失常被完全忽略。
- 誤區九：效益只算人力節省。品質提升的效益往往更大，且是說服決策者的關鍵。
- 誤區十：把安全預算當成可刪減的項目。它是法律義務，且風險評估必須依三階法的順序。
- 誤區十一：以為協作機器人不需要安全評估。它仍須經風險評估，且裝上工具後原評估即失效。
- 誤區十二：驗收通過就以為結束。維運成本常超過建置成本，可監測、可維護、可調整必須在設計階段就考慮。

課堂討論

1. 34.1.1 節指出「技術只佔失敗原因的一成」。請討論：這是否意味著技術能力不重要？一個技術強但不懂系統整合的工程師，與一個懂整合但技術普通的工程師，各自的價值在哪裡？
2. 例題 34-1 顯示「加一個倒角」勝過「花兩百萬提升精度」。請討論這個思維在本書其他章節的對應，並嘗試提出一個判準——什麼時候該改變問題、什麼時候該提升能力？
3. 34.6.3 節的知識框指出品質效益常大於人力效益。請討論：為何提案時人們仍傾向只算人力？這反映了什麼樣的認知偏誤？
4. 34.8.3 節提到台灣以中小企業為主，需要的是「彈性製造」而非「大批量專用自動化」。請討論這對機器人工程師的職涯選擇有什麼啟示？
5. 34.9.3 節說「上線後才是開始」。請討論：一個機器人系統要能穩定運轉三年，設計階段需要做哪些「當下看不出價值」的準備工作？

章末摘要

- 機器人專案的失敗原因中，技術只佔一成；需求不明（35%）與環境假設錯誤（25%）才是

主因。

- V 模型的核心是「每一層的分解都對應一層的驗證」；常見的錯誤是只做元件測試而不做系統測試。
- 客戶說的需求通常不是真實的需求；五問法能挖出真正的目的，並常找出更便宜的解法。
- 需求必須寫成可驗證的形式——有明確的數值、測試條件與樣本定義。
- 範圍潛變的三種控制方法：明確的範圍文件（含「不包含什麼」）、變更管理流程、分階段交付。
- 機器人工作單元中，本體與程式只佔約 30% 的工時；治具與訊號整合各佔 25% 以上。
- 介面問題有四類：時序、資料格式、握手協定、異常處理；其中異常處理最常被完全忽略。
- 核心原則：先問「能不能把環境變得可控」，再問「能不能用演算法處理不可控的環境」。
- POC 的價值不在證明可行，而在盡早發現不可行；必須用現場的真實條件測試。
- 能在 FAT 解決的問題，成本是 SAT 的十分之一；應請客戶參與 FAT。
- 誤差會在系統層級累積；各子系統達標不代表整體達標。
- 誤差預算的三種改善策略：提升精度（貴）、消除誤差源（中）、放寬容差（最便宜）。
- 稼動率的提升有兩條路——增加 MTBF 或降低 MTTR，後者往往便宜得多。
- $OEE = \text{稼動率} \times \text{性能效率} \times \text{良率}$ ；相乘的結構使「每一項都要很高」。
- 投資額常被低估的項目：整合工時、治具與周邊、以及產線停機損失。
- 效益不只是人力節省，還包含品質提升、產能提升與間接效益；品質效益常是最大的一項。
- 安全是法律義務而非品質項目；風險評估依三階段：本質安全 → 工程防護 → 資訊警告。
- 協作機器人仍須風險評估；裝上工具或抓取重物後，原有的安全評估即失效。
- 四大應用領域依環境結構化程度與任務變異度區分，技術重點與成熟度差距極大。
- 部署後最常見的失效不是程式錯誤，而是磨損、光源老化、來料變異等漸進性問題。
- 系統的價值不在交付當天多完美，而在三年後還在穩定運轉——需要可監測、可維護、可調整。

觀念題與計算題

觀念題

1. 列舉機器人專案失敗的五個主要原因及其佔比，並說明「技術只佔一成」的意義。
2. 說明 V 模型的結構與其核心洞見，並舉例說明「只做元件測試」的風險。
3. 說明五問法的作法，並解釋為何它常能找出更便宜的解法。
4. 比較「模糊的需求」與「可驗證的需求」，並各舉三個例子。
5. 說明需求規格中為何必須定義「測試的條件與樣本」。
6. 說明範圍潛變的成因與三種控制方法。
7. 列舉機器人工作單元的七類組成，並指出最常被低估的兩類。
8. 列舉介面問題的四類，並說明為何「異常處理」最常被忽略。
9. 說明「先讓環境可控」的原則，並舉三個用周邊設計降低技術難度的例子。
10. 說明治具方案的限制，以及在什麼情況下應改用視覺方案。
11. 說明 POC 的目的與三個原則，並解釋「早失敗比晚失敗好一百倍」。
12. 區分 FAT 與 SAT，並說明為何應請客戶參與 FAT。
13. 說明誤差為何會在系統層級累積，並比較最壞情況與均方根兩種計算方式。
14. 說明誤差預算的三種改善策略，並指出哪一種通常最划算。
15. 說明 MTBF、MTTR 與稼動率的關係，並解釋為何降低 MTTR 往往較便宜。
16. 說明 OEE 的三個組成，並解釋相乘結構的意義。
17. 列舉自動化投資成本中最常被低估的三個項目。
18. 列舉年度效益的四個類別，並說明為何品質效益常被忽略卻可能最大。
19. 說明機器人安全標準的三層體系，以及風險評估的五個步驟。
20. 說明風險減低的三階法及其優先順序，並解釋為何不可跳過第一階。
21. 說明協作機器人的四種模式，並解釋「協作不等於不需要安全評估」。
22. 比較四大應用領域的環境、任務變異、關鍵技術與成熟度。
23. 列舉部署後的六種常見失效模式，並說明哪兩種特別值得警惕。

計算題

1. 某任務容差 $\pm 0.3 \text{ mm}$ ，誤差來源為：機器人 ± 0.4 、視覺 ± 0.25 、手眼校正 ± 0.15 、工件公差 ± 0.2 、夾持 $\pm 0.15 \text{ mm}$ 。(a) 計算最壞情況與均方根總誤差。(b) 判斷是否可行。

(c) 若加裝定位治具可消除視覺與手眼誤差，重新計算。(d) 若改為在孔口加 0.8 mm 倒角，判斷是否可行並比較兩方案。

2. 某產線班別 8 小時，計畫性停機 45 分鐘，故障停機 30 分鐘。理論節拍 4.5 秒/件，實際生產 5,200 件，不良 130 件。(a) 計算稼動率。(b) 計算性能效率。(c) 計算良率。(d) 計算 OEE 並判斷是否達標竿水準。(e) 若把故障停機減半，計算新的 OEE。
3. 某自動化專案的成本為：機器人 120 萬、視覺 60 萬、治具周邊 80 萬、安全設備 30 萬、整合工時 130 萬、安裝試車 25 萬、安裝期停機損失 3 天（日產值 80 萬）。效益為：取代 4 人（每人年成本 58 萬）、不良率由 2.5% 降至 0.4%（月產 8 萬件、每件價值 300 元）。(a) 計算總投資額。(b) 計算年度效益（品質效益以五成計）。(c) 計算投資回收期。(d) 若只計人力節省，回收期為何？比較兩者並討論。
4. 某工位有三個自動化方案：甲投資 250 萬、年效益 200 萬、壽命 8 年；乙投資 480 萬、年效益 420 萬、壽命 8 年；丙投資 750 萬、年效益 600 萬、壽命 12 年。(a) 計算各方案的投資回收期。(b) 計算五年的累積淨效益。(c) 計算各自壽命期間的累積淨效益。(d) 綜合判斷應選哪一個，並列出你的判斷需要確認的三個前提。

動手做與專題延伸

本章實作：系統提案

本章的實作是撰寫一份完整的系統提案，詳見前面的「實作」一節。此處提供三個延伸的方向。

- **方向一：現場觀察與時間研究。**實際到一個有重複性工作的場所（工廠、餐廳廚房、圖書館），用碼表記錄每個動作的時間，繪出時間分布圖。**找出最耗時的三個動作——這就是自動化的優先目標。**
- **方向二：介面規格書的撰寫。**為你的專題中兩個子系統之間的介面，撰寫一份完整的介面規格書（34.3.2 節的知識框）。**特別要定義異常時的行為。**
- **方向三：安全風險評估。**對一個實際的機器人設備（可用實驗室的），依 34.7.3 節的五步驟做一次完整的風險評估，列出所有危害與防護措施。

專題延伸

- **需求釐清的實證研究：**找三位不同的人（如三位老師或三位同學），請他們各自描述「一台

自動整理實驗室的機器人應該做什麼」。記錄他們的原始描述，接著用五問法各訪談 20 分鐘，記錄挖掘出的真實需求。比較三份「表面需求」與「真實需求」的差異，並歸納出你發現的模式。這個練習能讓你親身體會 34.2 節的核心——**問題往往不是別人說的那個問題**。

- **誤差預算的實測驗證**：對實驗室的機器手臂做一次完整的誤差量測——重複精度（同一點來回 30 次）、絕對精度（用尺量測實際位置與指令的差）、以及溫度漂移（開機後每小時量一次，共 4 小時）。把實測值與規格書的數字比較，你很可能會發現絕對精度遠差於規格書上的重複精度（第 9.9.2 節）。接著用這些實測值做一次誤差預算，判斷你的專題是否可行。
- **進階：一個真實的產業案例研究**。聯繫一家有導入機器人的工廠（可透過學校的產學管道），實地訪談並記錄：他們當初為何導入、遇到什麼問題、實際的投資回收期與預期差多少、以及**如果重來一次會怎麼做**。最後一個問題往往能得到最有價值的答案。把訪談整理成一份案例報告，並用本章的框架（V 模型、誤差預算、OEE、投資評估）分析他們的決策。這個專題能讓你看見教科書與真實產業之間的落差，而那正是最寶貴的學習。

參考文獻與延伸閱讀

- INCOSE, Systems Engineering Handbook, 5th ed., Wiley（系統工程的權威指引，V 模型的標準定義）。
- Blanchard, B. S. and Fabrycky, W. J., Systems Engineering and Analysis, Pearson（系統工程的經典教材，含經濟分析）。
- ISO 12100:2010, Safety of Machinery — General Principles for Design（機械安全的基礎標準，風險評估的方法）。
- ISO 10218-1 & -2:2011, Robots and Robotic Devices — Safety Requirements for Industrial Robots（工業機器人與其系統整合的安全要求）。
- ISO/TS 15066:2016, Robots and Robotic Devices — Collaborative Robots（協作機器人的力與壓力限制）。
- ISO 13849-1:2015, Safety-Related Parts of Control Systems（安全控制系統的設計與 PL 等級）。
- Nakajima, S., Introduction to TPM: Total Productive Maintenance, Productivity Press（OEE 概念的出處）。
- Hitomi, K., Manufacturing Systems Engineering, Taylor & Francis（製造系統工程，含節

拍與產能分析) 。

- Groover, M. P., Automation, Production Systems, and Computer-Integrated Manufacturing, Pearson (自動化系統的完整教材，含經濟分析) 。
- 勞動部職業安全衛生署，〈機械設備器具安全資訊網〉與相關法規：台灣的機械安全法規與檢查基準。
- 經濟部產業發展署，〈智慧機械產業推動方案〉相關報告：台灣機器人與智慧製造的產業脈絡。
- International Federation of Robotics (IFR), World Robotics Report (年度出版)：全球機器人市場與應用趨勢的權威資料。

第 35 章

法規、商業化與社會接受度

編著：李世淵（ 機器人叫獸 ）

助教：李天宇、李宇晴

【章首情境圖建議】左半為餐廳現場的三格情境：第一格是送餐機器人與一位拄拐杖的長者在走道上正面相對、兩者都靜止，後方排了數人，旁註「技術上正確，結果是堵塞」；第二格是小孩伸手觸碰托盤、機器人急停、湯汁潑灑，旁註「急停不一定是最安全的反應」；第三格是餐廳老闆指著機器人提問，對話框寫著「如果它燙傷客人，是誰要賠？」。右半為上市路徑的階梯圖：由左下的「實驗室原型（TRL 5）」向右上經過場域試驗、設計定案、認證取得、小量試產，抵達「上市（TRL 9）」，並在「認證取得」處標註「3~9 個月、數十萬至數百萬元」。下緣文字：技術可行，不等於可以上市。

叫獸開講 「幫我把桌上的杯子拿到廚房」——這句話難在哪裡？

小矧的團隊做出了一台送餐機器人。技術上相當成功——SLAM 建圖穩定、避障靈敏、抓取平穩，在實驗室走廊上跑了三個月沒出過問題。他們決定去一家餐廳試營運。

第一天，機器人在走道上遇到一位拄拐杖的長者。它偵測到障礙、正確地停下、等待——然後就一直等下去。長者不知道該讓路，機器人不知道該繞行，後面排了五個人。

第二天，一個小孩把手伸向機器人的托盤。它偵測到有東西靠近，緊急停止——整盤湯灑了。

第三天，餐廳老闆問了一個問題，讓整個團隊愣住了：「如果它燙傷客人，是誰要賠？」

回到實驗室，叫獸把這三件事寫在白板上。「你們注意到了嗎？這三個問題，沒有一個是技術問題。」

小翰說：「第一個是.....不知道該怎麼跟人互動？」

「對。而且更深一層——它不知道社會的規則。人在走道上相遇時，會有一個微妙的協商：誰先動、誰讓路、要不要點頭示意。這套規則沒有寫在任何地方，但每個人都懂。」

「第二個呢？」小矧說：「它的反應是對的啊，偵測到人就停止。」

「技術上對，但結果是錯的。」叫獸說，「急停灑了湯，可能比慢慢閃開更危險。安全不是一個技術指標，而是一個情境判斷。」

「至於第三個問題.....」叫獸停頓了一下，「那才是最難的。因為它的答案不在你們的程式裡，而在法律、保險、與社會共識裡——而這三樣東西，目前都還沒準備好。」

他在白板上寫下這一章的主題：

技術可行 ≠ 可以上市。

「第 34 章教你們怎麼把技術變成一個能在工廠裡跑的系統。今天要教的是：當機器人走出工廠、走進有人的地方，還要跨過哪些門檻。」

本章為什麼重要

第 34 章的场景是工廠——一個高度可控的環境：進入者受過訓練、有圍籬隔離、有明確的

作業規範。但機器人的下一個戰場是**有人的空間**：餐廳、醫院、賣場、辦公室、以及家庭。

這個轉換帶來三類全新的挑戰，而它們都不是技術問題：

- **法規與認證**：工廠裡的機器人只需符合職安法規；但一台在公共空間運行的機器人，可能同時涉及電氣安全、電磁相容、無線電、個資保護、以及各地方政府的特別規定。**沒有這些文件，做得再好也不能賣。**
- **責任與保險**：工廠的操作員知道風險並受過訓練；但餐廳的客人沒有。一旦出事，責任歸屬（第 33.8.3 節）就成為真實的法律問題。
- **社會接受度**：一台技術完美但讓人不舒服的機器人，會被使用者拒絕——而這種拒絕往往是無聲的：他們不會抱怨，只會不用它。

本章的價值在於「**讓你知道有這些門檻存在**」。多數技術背景的團隊在做出原型後才發現，離「能賣」還有很長的路——而那條路的長度，往往超過開發本身。**提早知道，就能提早準備。**

學習目標

- 能說明工廠環境與公共空間對機器人要求的三類差異。
- 能說明產品標準與認證的體系及其在不同市場的差異。
- 能列舉一台服務機器人上市所需的主要認證項目。
- 能說明台灣、歐盟與美國在機器人法規上的主要差異。
- 能說明個資與隱私法規對機器人設計的具體要求。
- 能分析產品責任的歸屬並說明降低風險的三種手段。
- 能說明保險在機器人商業化中的角色。
- 能區分四種機器人的商業模式並比較其適用場合。
- 能說明技術成熟度與市場採用曲線的關係。
- 能分析社會接受度的四個構面並設計改善策略。
- 能說明人機互動設計的基本原則。
- 能為一個機器人產品規劃完整的上市路徑。

先備知識

第 34 章的系統整合與安全（本章是其在公共空間的延伸）、第 7 章的安全標準、第 33.8 ~ 33.9 節的社會影響與倫理、第 26.10.3 節的社會導航、第 20.10 節的協作安全。

本章沒有數學，但有大量的判斷與取捨。建議讀法：把每一節對照你熟悉的某個產品（如掃地機器人、送餐機器人），思考「它是怎麼處理這個問題的」。

核心名詞中英對照

表 35-1 本章核心名詞

中文	英文	說明
產品認證	Product Certification	由第三方確認符合標準的程序
符合性聲明	Declaration of Conformity	製造商自行聲明符合法規
CE 標誌	CE Marking	歐盟市場的強制性符合標誌
電磁相容	EMC	設備不干擾他人也不被干擾
低電壓指令	Low Voltage Directive	歐盟的電氣安全要求
型式認證	Type Approval	對產品型號的一次性認證
產品責任	Product Liability	產品缺陷造成損害的賠償責任
召回	Recall	產品有安全疑慮時的回收程序
個人資料保護	Personal Data Protection	對可識別個人資訊的法律保護
一般資料保護規則	GDPR	歐盟的個資保護法規
設計即隱私	Privacy by Design	在設計階段納入隱私保護
技術成熟度	Technology Readiness Level, TRL	技術由概念到量產的九級分類
市場採用曲線	Technology Adoption Curve	新技術被市場接受的階段模型
鴻溝	The Chasm	早期採用者與主流市場之間的斷層
機器人即服務	Robot-as-a-Service, RaaS	以訂閱制提供機器人服務
社會接受度	Social Acceptance	使用者與社會對技術的接納程度
恐怖谷	Uncanny Valley	過度擬人卻不完美引發的不適
社會可接受行為	Socially Acceptable Behavior	符合社會慣例的機器人行為

35.1 走出工廠

35.1.1 三類環境的對照

機器人的運行環境可分為三類，而**每跨越一類，難度就跳一個等級**：

表 35-2 三類運行環境的對照

面向	工廠（受控）	半公共（商業空間）	家庭與公共
周圍的人	受訓的員工	顧客、訪客	任何人（含幼童、長者）
環境的變異	低（固定佈局）	中（人流變動）	極高（無法預期）
隔離手段	圍籬、光柵	不可隔離	不可隔離
法規適用	職安法規	職安 + 公共安全	產品安全 + 消費者保護 + 個資
失敗的後果	停線、損壞	客訴、商譽	傷害、法律責任
維護支援	廠內有工程師	定期巡檢	使用者自行處理
可靠度要求	95~98%	98%+	99.9%+

最後一系列的差異特別值得注意。工廠裡稼動率 95% 是可接受的（每天停機一小時，有人會來修）；但一台家用機器人若每天故障一次，使用者會直接把它收進櫃子裡。

35.1.2 為何「不能隔離」改變了一切

工廠安全的核心手段是**隔離**（第 34.7.3 節的第二階工程防護）：用圍籬把人與機器分開。這個手段簡單、可靠、且成本低。

但在公共空間，**隔離是不可能的**——機器人存在的目的就是要在人的環境中活動。這使得安全必須改用另外兩種手段：

- **本質安全**（第一階）：限制速度、限制力量、圓角化外殼、降低質量。這是最可靠的手段（呼應第 20.10 節與第 33.3 節）。
- **主動偵測與反應**（第三階的延伸）：偵測人的存在並減速或停止。但如開頭故事所示，這不見得總是正確的反應。

◎ 急停不一定是最安全的反應

開頭故事的第二天，機器人偵測到小孩的手而緊急停止，結果湯灑了。這揭示了一個在工廠中不存在的問題：在有人的環境中，「停止」未必是最安全的選項。工廠的機器人停止就等於安全（因為人不該在那裡）；但服務機器人停在走道中央可能造成阻塞、急停可能潑灑液體、在樓梯口停止可能造成絆倒。正確的設計是「**情境化的安全反應**」——依當下的情況選擇減速、繞行、緩停或完全停止。這比單一的急停邏輯複雜得多，卻是進入公共空間的必要條件。

35.1.3 從「設備」到「產品」

還有一個法律上的關鍵轉變：工廠裡的機器人通常被視為**設備**（equipment），而消費或商用的機器人是**產品**（product）。這個區別有實質的後果：

表 35-3 設備與產品的法律差異

面向	設備（B2B 客製）	產品（量產銷售）
適用法規	職業安全衛生法規	商品檢驗法、消費者保護法
安全驗證	由整合商做風險評估	須通過型式認證
責任基礎	契約責任為主	產品責任（無過失責任）
使用者	受訓的操作員	一般消費者（不需受訓）
說明義務	操作手冊	警語、標示、使用說明
瑕疵時	維修或索賠	可能需召回

「無過失責任」是最關鍵的差異。在產品責任的架構下，即使製造商沒有過失，只要產品有缺陷造成損害，就要負賠償責任。這使得產品的安全設計必須遠比設備嚴謹。

35.2 標準與認證

35.2.1 標準與法規的區別

首先要釐清兩個常被混用的概念：

標準（Standard）：技術規範，通常是自願性的

法規（Regulation）：法律要求，強制性的

但兩者常有連結——法規常「引用」標準作為符合性的推定依據。

具體的運作方式是：法規說「產品必須安全」（原則），標準說「怎樣算安全」（具體要求）。若你的產品符合標準，通常就被推定符合法規。

35.2.2 一台服務機器人需要哪些認證

這是許多團隊在做出原型後才發現的清單。以一台在台灣販售的室內服務機器人為例：

表 35-4 服務機器人的主要認證項目

類別	項目	依據	說明
電氣安全	商品檢驗（BSMI）	商品檢驗法	強制性，未取得不得販售

類別	項目	依據	說明
電磁相容	EMC 測試	CNS 13438 等	確保不干擾他人設備
無線電	NCC 型式認證	電信管理法	含 WiFi、藍牙者皆須
電池	鋰電池安全	CNS 15364 等	電池是高風險項目
機械安全	風險評估報告	ISO 12100	第 34.7 節
服務機器人安全	ISO 13482	—	個人照護機器人的專用標準
個資	隱私政策與告知	個人資料保護法	35.4 節
(若含醫療宣稱)	醫材許可	醫療器材管理法	門檻極高，應避免醫療宣稱

最後一列是一個常見的陷阱。若行銷文案寫「可監測長者的健康狀況」，就可能被認定為醫療器材——而醫材的認證成本與時程是一般產品的十倍以上。因此許多產品刻意使用「生活紀錄」「活動提醒」等非醫療的用語。

35.2.3 ISO 13482：服務機器人的專用標準

值得特別介紹的是 ISO 13482——它是第一個專門針對個人照護機器人的國際安全標準（2014 年發布）。

它把服務機器人分為三類，各有不同的要求：

- 行動輔助機器人：協助行走、移位的設備。風險在於跌倒與夾擠。
- 個人載具機器人：載人移動的設備（如自動輪椅）。風險在於翻覆與碰撞。
- 人型服務機器人：在人的環境中執行任務。風險在於碰撞、夾擠與失控。

它與工業標準（ISO 10218）最大的差異是：工業標準假設「人不該在機器人附近」，因此重點是隔離；而 ISO 13482 假設「人必定在附近」，因此重點是限制傷害的程度——與第 20.10 節協作機器人的思維一致。

35.2.4 不同市場的差異

若要銷往國際，還必須面對各地的差異：

表 35-5 主要市場的認證體系

市場	體系	特點	難度
台灣	BSMI + NCC	商品檢驗 + 無線電型式認證	中

市場	體系	特點	難度
歐盟	CE 標誌	自我宣告 + 技術文件；機械指令、EMC 指令、RED	中高（文件要求嚴謹）
美國	FCC + UL/ETL	FCC 管無線電；UL 非強制但實務上必要	中高
日本	PSE + 技適	電氣用品安全法 + 無線電	高
中國	CCC + SRRC	強制性產品認證	高

一個實務的建議：若有國際市場的規劃，應在設計階段就把目標市場的要求納入。事後修改（如更換不合規的零件、重新佈線以通過 EMC）的成本，遠高於一開始就設計好。

◎ 認證的時間與成本，遠超過多數團隊的預期

一個常見的誤判是：「產品做好了，送測一下就好」。實務上，認證通常需要 3~9 個月與數十萬到數百萬元，且很少一次通過——EMC 不合格需要修改屏蔽或濾波、電氣安全不合格需要改變絕緣設計，每一次修改都要重新送測。因此認證應該在專案的時程與預算中被明確規劃，而非當成最後的一個小步驟。更好的作法是「預測試」：在開發中期就送去做一次非正式的 EMC 掃描，及早發現問題。

35.3 責任、保險與召回

35.3.1 產品責任的三種缺陷

法律上，產品缺陷分為三類，機器人三類都可能發生：

表 35-6 產品缺陷的三種類型

類型	定義	機器人的例子
設計缺陷	設計本身就有問題	夾點設計不當、速度上限過高
製造缺陷	設計正確但製造出錯	某批次的感測器焊接不良
警告缺陷	未充分告知風險與正確用法	未說明不可用於樓梯、未標示夾點

第三類最常被工程團隊忽略。一個技術上完美的產品，若因為說明書沒寫清楚而導致誤用受傷，製造商仍要負責。

35.3.2 降低責任風險的三種手段

手段一：設計上消除風險（最有效）。這是第 34.7.3 節三階段的第一階——能消除的危害就不要靠警告。例如：把夾點的間隙加大到手指伸不進去、限制最大速度、外殼圓角化。

手段二：完整的文件與可追溯性。第 33.8.3 節提過，實務的責任判斷傾向要求「**可追溯性與盡職義務**」。這意味著必須保存：

- **設計與測試紀錄：**證明做了合理的驗證。
- **風險評估報告：**證明辨識了危害並採取了措施。
- **運行日誌：**事故發生時能還原當時的狀況。這正是第 29.9.3 節「每次都錄 rosbag」的法律意義。
- **軟體版本紀錄：**能確認事故當時運行的是哪個版本。

手段三：明確的使用限制與警告。誠實地說明「這個產品不適合什麼情境」——如「不可用於有樓梯或落差的環境」「不可載運熱湯」「不建議在有幼童無人看管的空間使用」。

◎ 誠實的限制說明反而是優勢

許多團隊擔心「寫出限制會讓產品看起來很弱」。但實務經驗恰恰相反：**明確的限制說明能降低客戶的期待落差，減少客訴，也在法律上保護自己。**一台明確說「我只能在平坦地面運作」的機器人，客戶在鋪了地毯的地方使用出問題時，責任較輕；而一台宣稱「適用各種環境」的機器人，就必須為所有環境負責。這與第 34.2.4 節「明確寫出不包含什麼」是同一個道理——界定範圍不是示弱，而是專業。

35.3.3 保險的角色

保險是商業化中一個常被忽略但關鍵的環節。三種相關的保險：

- **產品責任險：**涵蓋產品缺陷造成第三人損害的賠償。多數 B2B 客戶會要求供應商投保——沒有它，可能連投標的資格都沒有。
- **公共意外責任險：**涵蓋在公共場所發生的意外。餐廳、賣場等場域通常要求。
- **營業中斷險：**若機器人故障造成客戶的營業損失。

一個實務的觀察：保險公司的核保會問很多問題——有沒有認證？有沒有風險評估？有沒有事故紀錄？**這些問題本身就是一份檢核表**，能幫助團隊發現自己的疏漏。而保費的高低，也反映了保險公司對這項技術風險的評估。

35.3.4 召回與軟體更新

機器人有一個傳統產品沒有的特性：**它的行為可以透過軟體更新改變**。這帶來機會也帶來風險。

機會：發現安全問題時，可以**遠端推送修正**，不需實體召回——成本可能相差百倍。

風險：

- **更新本身可能引入新問題。**一個推送到上千台機器人的錯誤更新，後果可能極為嚴重。
- **版本管理的複雜度：**現場可能有多個版本並存，事故時難以釐清。
- **法律上的模糊：**軟體更新是否算「修理」？更新後的產品責任如何認定？**這些在多數國家尚無明確的規範。**

因應之道：分批推送（先小範圍驗證）、保留回滾機制、完整記錄每台設備的版本歷程、以及更新前的完整回歸測試（第 29.9.3 節的 rosbag 在此再次派上用場）。

35.4 隱私與資料保護

35.4.1 機器人是一個移動的感測器

第 33.9.2 節已指出隱私問題的嚴重性，本節從**法規遵循**的角度補充。

關鍵的認識是：**機器人蒐集的多數資料，在法律上屬於個人資料：**

- **影像：**只要能辨識出個人（甚至只是背影加上時間地點），就是個資。
- **語音：**內容與聲紋皆是。
- **位置與軌跡：**能推知一個人的作息與行為模式。
- **環境地圖：**你家的格局本身就是敏感資訊。

35.4.2 個資法的核心要求

台灣的《個人資料保護法》與歐盟 GDPR 的核心要求相似，可歸納為五點：

表 35-7 個資保護的五項核心要求

原則	要求	機器人上的具體作法
告知	蒐集前須告知目的與範圍	首次使用時的說明、機身標示
目的限制	只能用於告知的目的	不可把導航用的影像拿去訓練模型 （除非另行同意）
最小化	只蒐集必要的資料	能在本地處理就不上傳、能用低解析度就不用高的
保存期限	目的達成後應刪除	設定自動刪除的週期
當事人權利	可查詢、更正、刪除	提供資料刪除的功能

第二列是最容易違反的一項。許多團隊會想「把使用者的資料拿來改善模型」——但若當初告知的目的只是「導航」，這就是違法的。正確的作法是明確告知並取得額外的同意。

35.4.3 設計即隱私

設計即隱私 (Privacy by Design) 是現代個資法規的核心原則：隱私保護必須在設計階段就納入，而非事後補救。

七個具體的設計原則：

- 本地優先：能在機器人上處理的就不要上傳（呼應第 32.10.1 節）。
- 去識別化：若必須上傳，先做模糊化或去除人臉。
- 明確的指示：以指示燈或聲音讓人知道相機何時運作。這也是社會接受度的關鍵（35.7 節）。
- 可控的禁區：讓使用者能指定「不進入、不記錄」的區域。
- 預設最保守：預設不蒐集、不上傳，由使用者主動開啟。
- 資料可攜與刪除：提供匯出與一鍵刪除的功能。
- 透明的政策：用一般人看得懂的語言說明蒐集了什麼、為什麼、存多久。

◎ 隱私設計不只是合規，更是市場的差異化

許多團隊把隱私當成「法務要求的麻煩」。但近年的市場趨勢顯示：隱私正在成為消費者的購買考量。一台明確標示「所有影像僅在本機處理、絕不上傳」的家用機器人，相對於一台把影像上傳雲端的競品，在行銷上是明確的優勢。而技術上，本地處理也帶來延遲低、不依賴網路的好處（第 32.10.1 節）。這是一個「合規、體驗、行銷」三者一致的少見案例——值得優先投入。

35.5 商業模式

35.5.1 四種模式

同樣的技術，用不同的商業模式銷售，會有截然不同的結果。機器人領域主要有四種模式：

表 35-8 四種機器人商業模式

模式	作法	優點	缺點	適用
設備銷售	一次買斷	現金流快、關係單純	客戶初期投資高、後續無收入	工業設備、已成熟的技術

模式	作法	優點	缺點	適用
系統整合	客製化專案 (第 34 章)	單價高、切中需求	難以規模化、依賴人力	產線自動化
機器人即服務 (RaaS)	訂閱制，按月/按件付費	降低客戶門檻、持續收入	初期資金壓力大、需長期維運	服務業、中小企業
平台與軟體	賣軟體或開發平台	邊際成本低、可規模化	需要生態系、競爭激烈	已有硬體生態的市場

35.5.2 RaaS 為何在服務機器人上特別重要

RaaS (Robot-as-a-Service) 是近年服務機器人的主流模式，原因有四：

- **降低採用門檻：**一台送餐機器人可能要價六十萬，對小餐廳是難以承受的投資；但每月一萬五的租金，與雇用一個工讀生相當，決策就容易得多。
- **風險轉移：**客戶不必承擔「買了不好用」的風險——不好用就不續約。這對尚未完全成熟的技術特別重要。
- **維運內含：**機器人需要定期保養與軟體更新 (第 34.9 節)。RaaS 把這些內含在服務中，客戶不需自行處理。
- **持續改善的誘因：**供應商的收入來自「客戶持續使用」，因此有強烈的動機去改善產品——這使雙方的利益一致。

但 RaaS 也有一個嚴峻的挑戰：現金流。設備的成本一次投入，收入卻分期回收：

設備成本 60 萬，月租 1.5 萬 → 需 40 個月才回本 (不計維運成本)

這對新創公司是巨大的資金壓力。

因應之道包括：與租賃公司合作 (由金融業承擔資產)、提高月租但降低合約期、或採「混合模式」 (部分買斷加部分服務費)。

35.5.3 定價的邏輯

一個常見的錯誤是用**成本加成法定價** (成本 $\times 1.5$ = 售價)。更好的方式是**價值定價**——依客戶得到的效益定價。

以送餐機器人為例：

客戶的效益 = 節省的人力 + 提升的翻桌率 + 減少的送錯率

若每月效益 4 萬，則月租 1.5 萬對客戶而言是明確划算的

定價的上限由客戶的效益決定，而非你的成本。

這也解釋了為何第 34.6.3 節的效益計算如此重要——你必須能清楚地向客戶說明「你會賺回多少」，才能支撐合理的價格。

◎ 一個定價的陷阱：與人力成本直接對標

服務機器人常被拿來與人力比較：「一台機器人月租 1.5 萬，比請一個人便宜」。但這個對標有風險——因為機器人做不到人能做的全部。一個服務生能送餐、能收桌、能回答問題、能處理客訴；而送餐機器人只能做第一項。若客戶期待「取代一個人」，必定失望。更好的定位是：「它承擔了最耗體力的來回搬運，讓你的服務生能專注在與客人的互動上」——這是「互補」而非「取代」（呼應第 33.8.1 節的三種影響）。這個定位不只更誠實，也更能被員工接受（他們不會覺得自己要被取代）。

35.6 技術成熟度與市場採用

35.6.1 技術成熟度等級 (TRL)

TRL (Technology Readiness Level) 是評估技術成熟度的標準框架，源自 NASA，現廣泛用於研發管理與補助審查：

表 35-9 技術成熟度等級 (TRL)

等級	階段	機器人領域的對應
TRL 1-3	基礎研究、概念驗證	論文、實驗室的原理驗證
TRL 4-5	實驗室與相關環境驗證	實驗室的完整原型 (多數學術研究止於此)
TRL 6-7	真實環境的原型驗證	現場試營運 (第 34 章的 POC 與 SAT)
TRL 8-9	完成認證、實際運作	通過認證、量產、商業運轉

一個關鍵的觀察：學術研究通常在 TRL 4-5 就發表論文並宣告成功；但商業化需要 TRL 8-9。而 TRL 5 到 8 的距離，往往比 1 到 5 更遠——這正是所謂的「死亡之谷」。

TRL 1→5：技術問題為主 (研究能力)

TRL 5→9：工程、認證、可靠度、成本 (完全不同的能力)

許多優秀的研究成果止步於此，不是因為技術不好，而是因為缺乏後半段所需的能力與資源。

35.6.2 市場採用曲線與鴻溝

即使技術成熟，市場的採用仍有其節奏。經典的**採用曲線**把使用者分為五類：

表 35-10 市場採用的五個階段

階段	佔比	特徵	對機器人的態度
創新者	2.5%	熱愛新技術本身	願意忍受不完美，主動嘗試
早期採用者	13.5%	尋求競爭優勢	願意冒險，重視效益
早期大眾	34%	務實，要看到成功案例	要求可靠、有支援、有參考客戶
晚期大眾	34%	保守，跟隨主流	等到成為標準配備才採用
落後者	16%	抗拒改變	被迫才採用

「鴻溝」(The Chasm) 出現在早期採用者與早期大眾之間——這是最多產品陣亡的地方。原因是兩者的需求截然不同：

早期採用者要的是：新的可能性 (願意忍受問題)

早期大眾要的是：可靠的解決方案 (不能有問題)

同一個產品往往無法同時滿足兩者。

跨越鴻溝的三個關鍵：

- **專注一個明確的利基**：與其做「通用服務機器人」，不如做「專門送餐的機器人」——把一件事做到極致可靠，建立可複製的成功案例。
- **建立完整的支援體系**：早期大眾要的不只是產品，還有安裝、訓練、維護、備品、以及「壞了有人來修」的保證 (第 34.9 節)。
- **可驗證的參考客戶**：「某某連鎖店用了兩年」比任何規格表都有說服力。

35.6.3 機器人領域的現況

用這兩個框架檢視機器人的各個領域 (呼應第 34.8.2 節)：

表 35-11 各領域的成熟度與市場位置

領域	TRL	市場階段	主要瓶頸
工業手臂	9	晚期大眾	已成熟，競爭在成本與彈性
掃地機器人	9	早期大眾 ~ 晚期大眾	已跨越鴻溝，滲透率持續上升
送餐/服務機器人	8	早期大眾 (正在跨越)	可靠度、與人互動、投資回收
倉儲揀貨	7-8	早期採用者 ~ 早期大眾	抓取的泛化 (第 21 章)

領域	TRL	市場階段	主要瓶頸
自駕車	7-8	早期採用者	長尾情境、法規、責任
人形機器人	4-6	創新者	可靠度、成本、實用性尚未證明
家用操作機器人	3-5	研究階段	靈巧操作 (第 33.7.3 節)

最後兩列值得特別注意。人形機器人近年獲得大量關注與投資，但其 TRL 仍在 4-6 之間——這意味著它距離「可靠地在真實環境中創造價值」仍有距離。這不是否定其潛力，而是提醒：關注度與成熟度是兩回事。

35.7 社會接受度

35.7.1 技術完美不等於被接受

開頭故事的第一天，機器人「正確地」停下來等待——技術上完全符合設計，但結果是走道被堵住、五個人在排隊、餐廳的服務體驗變差。

這揭示了一個關鍵的差異：

技術指標：偵測到障礙 → 停止 (正確)
社會期待：遇到人 → 禮讓、繞行、或示意 (自然)

兩者的落差，就是社會接受度的問題。

35.7.2 接受度的四個構面

使用者是否接受一台機器人，取決於四個構面：

表 35-12 社會接受度的四個構面

構面	使用者的問題	設計上的因應
有用性	「它真的幫到我了嗎？」	解決真實的痛點 (第 34.2 節的五問法)
易用性	「我會用嗎？會不會很麻煩？」	簡化操作、明確的回饋、容錯設計
信任	「它安全嗎？可預測嗎？」	行為的可預測性、明確的意圖表達
社會規範	「別人會怎麼看？我會不會很奇怪？」	符合場域的社會慣例、不引人側目

第三與第四構面最常被工程團隊忽略，卻往往是決定成敗的關鍵。

35.7.3 信任來自可預測性

一個反直覺的發現：使用者信任的不是「最聰明」的機器人，而是「最可預測」的機器人。

原因是：人需要能夠預期機器人接下來會做什麼，才能安心地與它共處。一台行為多變、難以預測的機器人（即使它每次的決策都更「最佳」），會讓人感到不安。

三個提升可預測性的設計：

- 行為的一致性：同樣的情境應該有同樣的反應。不要因為演算法的隨機性而讓行為變來變去。
- 意圖的表達：讓人知道它「打算做什麼」。這可以透過：轉向前先減速並轉頭（模仿人的行為）、用光條指示行進方向、用聲音提示「我要通過了」。
- 明確的狀態顯示：它現在是在工作、待命、還是故障？一個看不出狀態的機器人會讓人不知所措。

◎ 「腿部語言」：一個優雅的設計案例

有研究團隊在送貨機器人上做過一個實驗：加裝一個簡單的「眼睛」，在轉彎前先「看」向要去的方向。結果行人與機器人的碰撞事件顯著減少——不是因為機器人變聰明了，而是因為行人能預測它要往哪走，於是自然地讓開。這個設計的成本極低（兩個 LED 眼睛），效果卻超過複雜的路徑規劃改良。它體現了一個重要的原則：與其讓機器人更會閃避人，不如讓人更容易閃避機器人。這也是第 26.10.3 節社會導航的核心思想——溝通往往比計算更有效。

35.7.4 恐怖谷與外觀設計

恐怖谷（Uncanny Valley）是機器人外觀設計的重要現象：隨著擬人程度提高，人的好感度會先上升，但在「非常像人卻不完全像」的區間急遽下降，直到完全擬真才回升。

卡通化的外觀 → 好感度高

寫實但不完美的人臉 → 強烈的不適感 X

完全擬真 → 好感度回升（但技術上難以達成）

多數服務機器人刻意停留在第一區。

實務上的三個設計建議：

- 不要追求擬真。除非有極強的理由，否則卡通化或機械感的外觀更安全。
- 功能性的外觀勝過擬人。一台送餐機器人不需要臉——它需要的是穩固的托盤、清楚的方向指示、以及讓人一眼看出「它是機器」。

- **避免過度的擬人化承諾**。若外觀像人，使用者會期待它能像人一樣互動——而它做不到，於是失望。外觀應該誠實地反映能力。

最後一點是一個重要的設計倫理：外觀是一種承諾。一台看起來很聰明的機器人，若實際上很笨，使用者的失望會轉為排斥。

35.8 人機互動設計

35.8.1 從「操作」到「互動」

工業機器人的使用者是**受訓的操作員**，介面可以複雜（如示教器）。但服務機器人的使用者是**未受訓的一般人**——這使介面設計的邏輯完全改變。

表 35-13 兩種介面設計的對照

面向	工業（操作）	服務（互動）
使用者	受訓的技術人員	任何人
學習時間	數小時到數天	零（必須直覺）
介面複雜度	可以複雜	必須極簡
錯誤的容忍	使用者會小心	必須容錯
回饋方式	螢幕與指示燈	光、聲、動作、語音
設計重點	功能完整	可理解、可預測

35.8.2 多模態的回饋

由於使用者不會盯著螢幕，服務機器人必須用**多種管道**傳達資訊：

- **光**：狀態指示（待命/工作/故障）、方向指示（要往哪走）。光的優點是遠距可見且不吵。
- **聲**：提示音（「請讓一下」）、警示音。但要極為節制——一台一直發出聲音的機器人在餐廳裡會令人厭煩。
- **動作**：轉向前的減速、輕微的「點頭」示意。這是最自然的溝通方式（35.7.3 節的知識框）。
- **語音**：最直接但也最侵入。適合需要明確指示的場合，但在安靜的環境（如圖書館、醫院夜間）應避免。

一個實務原則：回饋的強度應與情境相稱。日常的狀態用光即可；需要人配合時用輕聲提示；

只有真正的危險才用警示音。

35.8.3 失敗的優雅處理

服務機器人一定會失敗——這是第 33.7.3 節「可靠度」問題的必然結果。因此**如何優雅地失敗**，比「不失敗」更務實。

四個原則：

- **及早求助勝過反覆嘗試**。若卡住了，與其嘗試二十次，不如在第三次後就請求協助。**反覆的失敗會讓旁觀者失去耐心**。
- **明確地說明狀況**。不要只顯示「錯誤 E-042」，而要說「我被擋住了，可以請你幫我移開嗎？」——**一般人看得懂的语言**。
- **保持安全的姿態**。失敗時應該停在不擋路、不危險的位置——**這需要在設計時就規劃「失敗時該退到哪裡」**。
- **讓人容易介入**。提供明確的手動模式、緊急停止、以及「推得動」的設計（斷電後可自由推動）。

◎ 一個容易被忽略的設計：讓它推得動

許多服務機器人在斷電或故障後**完全推不動**（因為減速機的自鎖，第 5.5 節）。這在實務上是嚴重的問題——一台故障的機器人卡在走道中央，**沒有人能移動它**，只能等工程師來。**因應之道有三**：使用可反向驅動的傳動（低減速比，第 5.5.2 節）、加裝手動釋放的離合器、或設計成可以整台抬起（控制總重量）。這是一個典型的「設計時想到就很簡單、事後才發現就很麻煩」的細節——也呼應了第 34.9 節「可維護性必須在設計階段考慮」。

35.8.4 與現場人員的關係

最後一個常被忽略的面向：**機器人不只服務顧客，還要與現場的員工共處**。

若員工不接受它，**機器人就會被「架空」**——他們會找理由不用它、或在它出問題時不願處理。三個實務的建議：

- **讓員工參與導入的過程**。在規劃階段就聽他們的意見——**他們最清楚現場的實際狀況**（呼應第 34.2 節的需求訪談）。
- **明確定位為「輔助」而非「取代」**（35.5.3 節的知識框）。強調它承擔的是最累的部分。
- **讓他們能掌控它**。提供簡單的手動操作、暫停、與呼叫功能。**能掌控的東西才不會被視為威脅**。

脅。

這也呼應第 33.8.2 節的自動化悖論：系統越自動化，人的角色越關鍵。而一個被員工排斥的系統，在需要人介入時就會失敗。

35.9 上市的完整路徑

35.9.1 六個階段

整合前面各節，一個機器人產品從原型到上市的完整路徑為：

表 35-14 產品上市的六個階段

階段	主要工作	常見的時程	關鍵風險
一、原型驗證	技術可行性 (TRL 5-6)	6~18 個月	技術做不出來
二、場域試驗	真實環境的試營運	3~9 個月	實際情境與預期不符
三、設計定案	為量產與認證而重新設計	3~6 個月	低估重新設計的工作量
四、認證取得	各項法規認證 (35.2 節)	3~9 個月	測試不過需修改再送
五、小量試產	驗證製造與品質	2~4 個月	良率、供應鏈
六、上市與維運	銷售、支援、持續改善	持續	維運能量不足

第三階段最常被低估。一個實驗室的原型與一個可量產的產品，差異極大：

- **零件的取得性**：實驗室可以用單價五千的實驗級零件，量產必須找到穩定供應且成本可行的替代品。
- **製造的可行性**：3D 列印的外殼在量產時要改為射出成型——而射出成型有拔模角、肉厚、分模線等一堆限制。
- **維修的設計**：實驗室的機器壞了就拆開修；產品必須考慮誰來修、怎麼拆、要不要防止使用者自行拆解。
- **成本的壓縮**：原型的物料成本可能是目標售價的三倍——需要大幅的重新設計。

35.9.2 場域試驗：最有價值的一步

第二階段的場域試驗，其價值等同於第 34.4.2 節的 POC——它會暴露所有你想不到的問題。

開頭故事的三天，正是場域試驗的縮影：

- **第一天的問題（不知如何與人協商）**：這在實驗室永遠測不出來，因為實驗室裡沒有拄拐杖的長者。
- **第二天的問題（急停灑湯）**：這需要真實的液體、真實的小孩、真實的意外。
- **第三天的問題（責任歸屬）**：這只有真正的營運者才會問。

場域試驗的三個原則：

- **盡早開始，即使產品不完美**。等到「做好了再去試」，通常已經投入太多而難以改變設計方向。
- **觀察勝過詢問**。使用者說「還不錯」但實際上很少用它——**行為比言語誠實**。應記錄實際的使用頻率與情境。
- **記錄所有的意外**。每一個「怎麼會這樣」都是一個設計的缺口。**這些累積起來，就是產品與競品的差距**。

35.9.3 一份上市檢核表

以下是一份實用的檢核表，涵蓋本章的所有面向：

表 35-15 機器人產品上市檢核表

類別	檢核項目	本章節次
安全	完成風險評估報告	第 34.7 節
	符合適用的安全標準（ISO 13482 等）	35.2.3
	失效時的行為安全且不阻塞	35.8.3
認證	電氣安全、EMC、無線電認證	35.2.2
	確認未涉及醫療或其他特許宣稱	35.2.2
	目標市場的在地認證需求已確認	35.2.4
責任	投保產品責任險	35.3.3
	使用限制與警告已明確標示	35.3.2
	運行日誌可追溯	35.3.2
隱私	隱私政策已撰寫並告知	35.4.2
	資料最小化與本地處理已實作	35.4.3
	提供資料刪除的功能	35.4.2
互動	非技術人員能零學習使用	35.8.1

類別	檢核項目	本章節次
	行為可預測、意圖可辨識	35.7.3
	斷電後可推動或有手動釋放	35.8.3
維運	備品清單與供應鏈已確認	第 34.9 節
	遠端診斷與更新機制已建立	35.3.4
	現場人員的訓練材料已備妥	35.8.4

35.10 台灣的機會與策略

35.10.1 三個結構性的優勢

第 34.8.3 節談過台灣的產業脈絡，本節從**機器人產品化**的角度補充三個優勢：

優勢一：完整的硬體供應鏈。從機構加工、電子組裝、馬達、到模具，台灣的供應鏈密度極高且反應快速。這使得「快速試錯」的成本遠低於其他地區——一個機構件的修改，在台灣可能三天完成，在其他地方可能三週。

優勢二：ICT 與機械的交會。機器人正好需要這兩者的結合，而台灣同時具備兩個產業的深厚基礎。這在全球是少見的組合。

優勢三：高齡化與缺工的內需市場。台灣的高齡化速度居世界前列，加上服務業的缺工——這創造了真實的需求，而非想像的需求。且台灣市場的規模適中，適合作為驗證的場域（不像大市場那樣競爭激烈，也不像小市場那樣沒有規模）。

35.10.2 三個結構性的挑戰

但也必須誠實面對三個挑戰：

- **內需市場規模有限：**單靠台灣市場難以支撐大規模的研發投入，必須及早規劃國際化——而這又回到 35.2.4 節的認證問題。
- **軟體與系統整合的人才相對稀缺：**台灣的強項在硬體製造，但機器人的價值越來越多在軟體與系統。
- **品牌與通路的經驗不足：**長期的代工模式使得「做出好產品」與「賣出好產品」之間有落差。

35.10.3 一個務實的策略建議

綜合以上，對台灣的機器人團隊，有三個務實的建議：

建議一：從利基市場切入，而非通用市場。與其做「通用服務機器人」與國際大廠競爭，不如專注一個明確的場域（如特定類型的餐廳、特定的檢測任務），**把它做到極致可靠**。這也符合 35.6.2 節「跨越鴻溝」的原則。

建議二：善用供應鏈優勢做快速迭代。台灣最大的優勢是「改得快」——**應該把這個優勢用在場域試驗的快速迭代上**（35.9.2 節），而非追求一次到位的完美設計。

建議三：把系統整合能力視為核心競爭力。第 34 章指出整合佔專案的 30~50%，而這正是難以被複製的能力——它需要對現場的深刻理解，而這是本地團隊的天然優勢。

35.11 由上市邁向專題

35.11.1 本章的成果

本章補完了「技術之外」的最後一塊。你現在能夠：說明公共空間與工廠環境的三類差異、列舉服務機器人所需的主要認證、分析產品責任的歸屬與降低手段、依個資法規設計隱私保護、比較四種商業模式並理解 RaaS 的邏輯、用 TRL 與採用曲線評估技術的位置、設計提升社會接受度的互動、以及規劃完整的上市路徑。

最重要的是三個認識：

- **技術可行 ≠ 可以上市。**認證、責任、保險、社會接受度——每一項都可能是致命的門檻，且都需要時間與成本。
- **可預測性勝過聰明。**使用者信任的不是最聰明的機器人，而是最可預測的機器人。
- **外觀是一種承諾。**它應該誠實地反映能力——過度的擬人化只會帶來失望與排斥。

35.11.2 接下來：把知識變成你自己的作品

第 36 章是全書的最後一章，也是所有內容的最終整合——**完整的專題方法論**。

它會帶你走過一個完整的專題：從**選題**（如何找到一個「值得做且做得完」的題目）、**規劃**（如何在有限的時間內完成）、**開發**（如何有效地推進與除錯）、**測試**（如何驗證你真的做到了）、到**發表**（如何讓別人理解你的價值）。

而它會特別著重一件事：失敗原因的分析與設計改良。因為在真實的專題中，失敗是常態而非例外——而能否從失敗中學到東西，決定了這個專題對你的價值。

換句話說：前三十五章教你**機器人學是什麼**，第 36 章教你**如何用它做出你自己的東西**。

完整案例分析

本章沒有計算例題，取而代之的是三個貼近實務的案例分析。

案例 35-1 送餐機器人的上市評估

情境：某團隊完成了送餐機器人的原型，在實驗室運行穩定。他們計畫六個月後上市，售價 55 萬元/台。請評估這個計畫的可行性。

分析一：時程的檢視 (35.9.1 節)

目前的狀態約在 TRL 5-6 (實驗室原型)。要達到可上市的 TRL 8-9，仍需經過：

場域試驗 3~9 個月 + 設計定案 3~6 個月 + 認證 3~9 個月 + 試產 2~4 個月 = 最快 11 個月，一般 18~28 個月

六個月的計畫明顯過於樂觀。

最可能的結果：趕在六個月上市，但跳過了場域試驗與完整認證——這會導致上市後大量的問題與潛在的法律風險。

分析二：認證的檢視 (35.2.2 節)

這台機器人含 WiFi、鋰電池、且在公共場所運行，因此至少需要：

- BSMI 商品檢驗 (電氣安全) ——強制，未取得不得販售。
- NCC 型式認證 (無線電) ——強制。
- 鋰電池安全測試——高風險項目，測試較嚴格。
- EMC 測試——馬達驅動器是常見的干擾源，首次測試不過的機率很高。

預估：認證費用 60~150 萬、時程 4~8 個月 (含一次修改重測)

這筆預算與時間必須明確納入計畫。

分析三：商業模式的檢視 (35.5 節)

售價 55 萬對小餐廳而言是很高的門檻。以一家月營收 200 萬的餐廳而言，55 萬是近一個月的營收——決策層級會拉高到老闆，決策周期會拉長。

建議改為 RaaS：

月租 1.8 萬（約等於一個工讀生的成本）
設備成本 30 萬 → 回本需 17 個月（不計維運）
三年合約的總收入 = 64.8 萬 > 買斷的 55 萬

對雙方都更有利：客戶門檻降低、供應商總收入更高且有持續關係。

但要注意現金流：若第一年要部署 50 台，需要 $30 \times 50 = 1,500$ 萬的資金投入，而第一年的收入只有約 1,080 萬。這個資金缺口必須事先規劃（35.5.2 節）。

綜合建議：

1. 把時程調整為 18 個月，並明確規劃各階段。
2. 立即開始場域試驗（不要等產品完美），與 2~3 家餐廳合作。
3. 同步啟動認證的預測試（尤其是 EMC），及早發現問題。
4. 改採 RaaS 模式，並規劃資金來源（自有資金、租賃公司合作、或投資）。
5. 投保產品責任險，並在合約中明確界定責任範圍。

案例 35-2 三天的問題該如何解決

情境：回到開頭故事的三個問題。請為每一個提出具體的設計解法。

問題一：與拄拐杖長者的僵局

問題的本質：機器人只有「停止」與「前進」兩種行為，缺乏「協商」的能力。

三層解法：

- 表達意圖（35.7.3 節）：加裝方向指示燈，讓長者知道機器人打算往哪走。這是成本最低也最有效的一步。
- 主動溝通：偵測到僵局超過三秒時，發出輕聲的「不好意思，我從您右邊過去」，同時慢速地往右移動——用行動配合語言。
- 繞行策略：若走道寬度足夠，主動貼邊繞行而非正面停止。這需要第 26 章的局部規劃器支援「側向繞行」而非只有「停止」。

問題二：急停造成潑灑

問題的本質：把「安全」等同於「停止」（35.1.2 節的知識框）。

三層解法：

- **機構層（最可靠）**：加裝托盤的防潑設計——凹槽、防滑墊、或加蓋。這是第 33.3 節形態計算的思維：與其讓控制更聰明，不如讓機構容錯。
- **控制層**：把「急停」改為「緩停」——依偵測到的距離決定減速率。只有在極近距離（如 20 公分內）才急停。
- **規劃層**：降低整體速度上限，並在人多的區域自動減速（第 26.7 節 DWA 的速度限制）。

注意三層解法的優先順序：機構層最可靠且不會失效，應優先採用。

問題三：燙傷的責任歸屬

這個問題沒有技術解，但有工程與商業上的因應（35.3 節）：

- **設計上降低風險**：限制可載運液體的溫度（如加裝溫度感測，超過 60°C 拒絕載運）、要求加蓋、限制加速度。
- **明確的使用限制**：在合約與說明書中寫明「不建議載運超過 60°C 的液體」——這是警告缺陷的防範（35.3.1 節）。
- **投保產品責任險**（35.3.3 節）。
- **合約中界定責任**：明確約定餐廳方的責任（如確保走道淨空、確保容器加蓋）與供應商的責任（設備的正常運作）。
- **運行日誌**：記錄事故當時的速度、感測資料、與控制指令——這是釐清責任的關鍵證據（35.3.2 節）。

一個誠實的結論：這個問題無法完全消除風險，只能降低機率並明確分擔。而這正是所有真實產品的處境——沒有零風險的產品，只有風險被合理管理的產品。

案例 35-3 隱私設計的取舍

情境：一台居家陪伴機器人需要辨識家中成員以提供個人化服務。團隊面臨三個設計選項，請分析其取舍。

表 35-16 三種隱私設計方案的比較

方案	作法	隱私	功能	成本
甲：雲端辨識	影像上傳雲端做人臉辨識	最差	最強（可用大模型）	低（不需高階硬體）
乙：本地辨識	在機器人上做人臉辨識	佳	中（受限於本地算力）	高（需 AI 加速硬體）
丙：非視覺辨識	用語音或穿戴裝置辨識	最佳（不拍攝）	弱（需主動配合）	低

分析：

- 方案甲功能最強但隱私最差。在家庭場景中，這可能是致命的——使用者對「家中的攝影機把影像傳到雲端」有強烈的疑慮（第 33.9.2 節）。
- 方案乙是多數產品的選擇。成本較高，但「所有影像僅在本機處理」是強而有力的行銷訴求（35.4.3 節的知識框）。
- 方案丙隱私最好但功能受限。適合作為輔助而非主要方案。

建議：以方案乙為主，方案丙為輔。具體的設計為：

1. 本地辨識：人臉辨識完全在機器人上執行，影像絕不上傳。
2. 只儲存特徵而非影像：辨識完成後即刪除原始影像，只保留無法還原的特徵向量。
3. 明確的指示：相機運作時亮起明確的指示燈。
4. 可控的禁區：讓使用者設定「不進入、不辨識」的房間（如臥室、浴室）。
5. 一鍵刪除：提供清除所有個人資料的功能。
6. 保留方案丙作為備援：若使用者選擇關閉視覺辨識，可改用語音辨識——把選擇權交給使用者。

核心的判斷：在家庭場景中，隱私不是一個可以用功能交換的項目。一個功能稍弱但令人安心的產品，勝過一個功能強大但讓人不敢放在客廳的產品。

實作：產品上市的規劃

本章的實作是為一個機器人產品（可用你的專題）規劃完整的上市路徑。

1. 定位分析：用 35.6 節的框架，評估你的產品目前在 TRL 的哪一級、目標客戶屬於採用曲線

的哪一群。**誠實地評估。**

2. **環境分析**：用表 35-2 判斷你的產品要在哪一類環境運行，並列出因此增加的要求。
3. **認證清查**：列出你的產品需要的所有認證（35.2.2 節）。上網查詢至少三項的實際費用與時程——你可能會被數字嚇到，而這正是重點。
4. **責任分析**：列出你的產品可能造成的三種傷害，並依 35.3.1 節分類（設計/製造/警告缺陷）。為每一項提出防範措施。
5. **隱私盤點**：列出你的產品會蒐集的所有資料，判斷哪些屬於個資，並依 35.4.3 節的七原則設計保護措施。
6. **商業模式設計**：比較買斷與 RaaS 兩種模式，計算各自的客戶成本與你的現金流。**特別要算出 RaaS 的資金缺口。**
7. **接受度評估**：用 35.7.2 節的四個構面，評估你的產品在每一項的表現。**找出最弱的一項並提出改善。**
8. **互動設計檢視**：檢查你的產品是否符合 35.8 節的原則——非技術人員能否零學習使用？行為是否可預測？失敗時是否優雅？斷電後推得動嗎？
9. **場域試驗規劃**：設計一個場域試驗的計畫——去哪裡、待多久、觀察什麼、如何記錄。**重點是「觀察勝過詢問」。**
10. **填寫上市檢核表**：用表 35-15 逐項檢視，標出「已完成」「進行中」「尚未開始」。
11. **撰寫上市計畫書**：整合以上為一份 8~12 頁的計畫書，包含：產品定位、目標市場、上市時程、認證計畫、商業模式、風險與對策。
12. **（進階）實地訪談**：找一位潛在的使用者（如餐廳老闆、長照機構主管），呈現你的計畫並訪談。**特別問他們「你最擔心什麼」——那往往是你沒想到的。**

國中小也看得懂：做得出來不等於賣得出去

假設你和同學發明了一台超厲害的機器人，可以幫忙送東西。你們很興奮，想把它拿去賣。

但等一下——做出來，跟可以拿去賣，是兩回事喔！

先想想這幾個問題：

問題一：它會不會電到人？ 你必須送去給專門的機構檢驗，證明它是安全的。這個檢驗要

花好幾個月、好幾十萬。

問題二：它的 WiFi 會不會干擾別人？也要送檢驗。

問題三：如果它撞到人，誰要賠錢？ 是做的人？賣的人？還是用的人？這要看合約怎麼寫、有沒有保險。

問題四：它拍到的照片存在哪裡？ 如果傳到網路上，那就是別人的隱私。法律規定你必須先告訴人家、而且用完要刪掉。

有沒有發現？**這些問題都跟「機器人做得好不好」沒關係，但每一個都可能讓你賣不出去。**

還有一個更有趣的問題：**就算全部都過關了，人家會喜歡它嗎？**

有一個真實的故事：一台送餐機器人在走道上遇到一位拿拐杖的老爺爺。機器人很「乖」地停下來等——結果**兩個都不動，後面排了一長串人。**

為什麼？因為人跟人相遇的時候，會有一種默契：誰先讓、誰先走、要不要點個頭。**這種默契沒有寫在任何地方，但每個人都懂——只有機器人不懂。**

後來工程師想到一個很妙的辦法：**在機器人上裝兩個像眼睛的燈，轉彎前先「看」向要去的方向。**結果撞到人的次數大幅減少了！

不是因為機器人變聰明了，而是因為**人可以猜到它要往哪走，就自己讓開了。**

這就是這一章最重要的一句話：與其讓機器人更會躲人，不如讓人更容易躲機器人。

所以啊，做機器人不只要懂技術，**還要懂人。**這可能是最難、但也最有趣的部分。

叫獸提醒與常見錯誤

- 誤區一：以為做出原型就快上市了。TRL 5 到 8 的距離往往比 1 到 5 更遠，這正是「死亡之谷」。
- 誤區二：把認證當成最後的小步驟。它通常需要 3~9 個月與數十萬元，且很少一次通過，必須納入時程與預算。
- 誤區三：行銷文案使用醫療宣稱。這可能使產品被認定為醫療器材，認證成本暴增十倍以上。

- 誤區四：以為「技術上正確」就是安全。急停可能潑灑液體、停在走道可能造成阻塞——安全是情境判斷。
- 誤區五：忽略警告缺陷。技術完美的產品若說明不足導致誤用，製造商仍要負責。
- 誤區六：擔心寫出使用限制會讓產品看起來很弱。明確的限制能降低期待落差、減少客訴，並在法律上保護自己。
- 誤區七：把導航蒐集的影像拿去訓練模型。若當初告知的目的只是導航，這就違反了目的限制原則。
- 誤區八：用成本加成法定價。應以客戶的效益定價，而效益的計算必須完整（第 34.6.3 節）。
- 誤區九：把機器人定位為「取代一個人」。它做不到人的全部，應定位為承擔最耗體力的部分。
- 誤區十：追求最聰明而非最可預測。使用者信任的是行為一致、意圖明確的機器人。
- 誤區十一：追求擬真的外觀。恐怖谷效應使「像人卻不完美」引發強烈不適，且外觀是一種能力承諾。
- 誤區十二：忘記設計「斷電後推得動」。一台推不動的故障機器人會卡在走道上，只能等工程師來。

課堂討論

1. 開頭故事的三個問題中，第三個（責任歸屬）沒有技術解。請討論：工程師在面對「無法用技術解決的問題」時，應該扮演什麼角色？
2. 35.7.3 節的知識框顯示「兩個 LED 眼睛」勝過複雜的路徑規劃改良。請討論這個現象，並嘗試找出本書其他章節中類似的「低成本高效益」設計。
3. 35.5.3 節主張機器人應定位為「互補」而非「取代」。請討論這個定位的誠實性與商業性之間是否有衝突，以及如何平衡。
4. 35.4.3 節的知識框指出隱私是「合規、體驗、行銷三者一致」的少見案例。請討論還有哪些設計決策具備這種特性，以及為何它們如此稀少。
5. 35.6.3 節指出人形機器人的 TRL 僅在 4-6，卻獲得大量關注與投資。請討論「關注度」與「成熟度」脫節的原因與後果。

章末摘要

- 三類環境（工廠、半公共、家庭公共）的要求逐級提高，可靠度需求由 95% 提升到 99.9%。
- 公共空間不可用隔離作為安全手段，必須改用本質安全與情境化的反應。
- 急停不一定是安全的反應——在有人的環境中，停止可能造成阻塞或潑灑。
- 從「設備」變成「產品」帶來法律上的關鍵轉變：適用產品責任的無過失責任。
- 標準是技術規範（自願），法規是法律要求（強制），兩者常有引用關係。
- 服務機器人的主要認證：電氣安全、EMC、無線電、電池、機械安全、個資；應避免醫療宣稱。
- ISO 13482 是個人照護機器人的專用標準，其思維是「假設人必定在附近」。
- 認證通常需要 3~9 個月與數十萬到數百萬元，且很少一次通過；應做預測試。
- 產品缺陷分三類：設計、製造、警告；第三類最常被工程團隊忽略。
- 降低責任風險的三種手段：設計上消除、完整的可追溯文件、明確的使用限制。
- 誠實的限制說明能降低期待落差、減少客訴，並在法律上保護自己。
- 機器人蒐集的影像、語音、位置與環境地圖，多數在法律上屬於個人資料。
- 個資保護的五項核心：告知、目的限制、最小化、保存期限、當事人權利。
- 設計即隱私的七原則中，「本地優先」同時滿足合規、體驗與行銷三個目的。
- 四種商業模式中，RaaS 因降低採用門檻與利益一致而成為服務機器人的主流，但有現金流壓力。
- 定價應依客戶的效益而非成本加成；且應定位為互補而非取代。
- TRL 5 到 8 的距離往往比 1 到 5 更遠，這是「死亡之谷」的成因。
- 採用曲線的鴻溝出現在早期採用者與早期大眾之間；跨越需要專注利基、完整支援與參考客戶。
- 社會接受度的四個構面：有用性、易用性、信任、社會規範；後兩者最常被忽略。
- 使用者信任的不是最聰明的機器人，而是最可預測的機器人。
- 恐怖谷效應使擬真但不完美的外觀引發不適；且外觀是一種能力的承諾。

- 服務機器人必用多模態回饋（光、聲、動作、語音），且回饋強度應與情境相稱。
- 失敗是常態，優雅的失敗處理包含：及早求助、明確說明、安全姿態、讓人容易介入。
- 機器人不只服務顧客，還要與現場員工共處；被員工排斥的系統會被架空。

觀念題與分析題

觀念題

1. 比較工廠、半公共與家庭公共三類環境的七個面向差異。
2. 說明為何「不能隔離」根本地改變了公共空間機器人的安全設計。
3. 解釋為何「急停不一定是最安全的反應」，並舉三個例子。
4. 比較「設備」與「產品」的六個法律差異，並說明無過失責任的意義。
5. 區分標準與法規，並說明兩者的連結方式。
6. 列舉一台台灣販售的服務機器人所需的主要認證項目。
7. 說明 ISO 13482 與 ISO 10218 在思維上的根本差異。
8. 說明認證的典型時程與成本，以及「預測試」的價值。
9. 列舉產品缺陷的三種類型，並說明為何警告缺陷最常被忽略。
10. 說明降低責任風險的三種手段，並解釋運行日誌的法律意義。
11. 說明為何「誠實的限制說明」反而是優勢。
12. 列舉三種與機器人相關的保險，並說明保險核保的附帶價值。
13. 說明軟體更新帶來的機會與三個風險。
14. 列舉機器人蒐集的四類個人資料，並說明環境地圖為何敏感。
15. 說明個資保護的五項核心要求，並指出最容易違反的一項。
16. 列舉設計即隱私的七個原則，並說明「本地優先」為何是三贏。
17. 比較四種商業模式，並說明 RaaS 在服務機器人上的四個理由。
18. 說明 RaaS 的現金流挑戰與三種因應方式。
19. 說明價值定價的邏輯，以及「與人力成本對標」的陷阱。
20. 說明 TRL 的分級，以及「死亡之谷」的成因。

21. 說明採用曲線的五個階段，以及鴻溝出現的原因與跨越方法。
22. 列舉社會接受度的四個構面，並說明哪兩個最常被忽略。
23. 解釋「使用者信任的是可預測而非聰明」，並列舉三個提升可預測性的設計。
24. 說明恐怖谷效應與三個外觀設計建議，並解釋「外觀是一種承諾」。
25. 列舉優雅失敗處理的四個原則，並說明「讓它推得動」為何重要。

分析題

1. **上市評估**：為你的專題（或一個你熟悉的機器人產品）做完整的上市評估。包含：TRL 定位、所需認證清單與預估時程費用、三種可能的產品缺陷與防範、以及建議的商業模式。**特別要誠實地評估「距離可上市還有多遠」。**
2. **安全反應的設計**：針對「服務機器人在走道上遇到人」這個情境，設計一套完整的反應邏輯。要求：（a）至少三個層級的反應（依距離或風險）；（b）每個層級都要說明「為何這個反應在此情境下是最適當的」；（c）考慮至少三種特殊情況（長者、幼童、載運液體時）。
3. **隱私設計的取舍**：為一台居家機器人設計隱私保護方案。列出它會蒐集的所有資料、判斷哪些是個資、並為每一項設計保護措施。接著分析：若要提供「跌倒偵測」的功能（需要持續的影像分析），該如何在功能與隱私之間取舍？
4. **社會接受度的改善**：選一台你觀察過的服務機器人（餐廳、賣場、機場皆可），用 35.7.2 節的四個構面評估它，找出最弱的一項。**接著提出三個具體的改善設計**，並說明每一個的成本與預期效果。

動手做與專題延伸

本章實作：上市規劃

本章的實作是為一個機器人產品規劃完整的上市路徑，詳見前面的「實作」一節。此處提供三個延伸方向。

- **方向一：認證的實地調查**。實際致電或查詢一家檢測機構（如台灣電子檢驗中心、SGS），詢問一台含 WiFi 的服務機器人所需的認證項目、費用與時程。**把真實的數字記錄下來——這會比任何教科書的描述更有說服力。**
- **方向二：使用者觀察**。到一個有服務機器人的場所（餐廳、賣場），觀察至少一小時。記

錄：人們如何與它互動、發生了幾次尷尬或衝突、店員如何處理異常。用 35.7.2 節的四個構面評分。

- **方向三：隱私政策的撰寫。**為你的專題撰寫一份使用者看得懂的隱私政策（不超過一頁）。用一般人的語言說明：蒐集什麼、為什麼、存多久、如何刪除。

專題延伸

- **服務機器人的接受度研究：**設計一份問卷或訪談提綱，調查一般人對服務機器人的態度。**關鍵是要區分「他們說的」與「他們做的」**——除了問卷，也要實地觀察他們的實際行為。你很可能會發現落差（如問卷說「很願意使用」但實際上會繞開它）。分析這個落差的成因，並提出設計上的改善。
- **意圖表達的實驗：**實作 35.7.3 節知識框中的「機器人眼睛」概念。在你的移動機器人上加裝方向指示（LED 或簡單的螢幕），設計一個實驗：讓機器人在走道上與行人相遇，分別在「有指示」與「無指示」兩種條件下各測 30 次，記錄：僵局發生的次數、行人讓路的比例、平均通過時間。**這個實驗成本極低但結果往往很驚人。**
- **進階：完整的產品化規劃。**選一個你的專題，做一份完整的產品化規劃書（20~30 頁），涵蓋本章與第 34 章的所有面向：需求與定位、技術架構、認證計畫（含實際查詢的費用與時程）、責任與保險、隱私設計、商業模式與財務試算、上市時程、風險與對策。**接著找一位業界人士（可透過學校的產學管道）審閱並給予意見。**這份文件不只是作業——它可能是你創業或求職時最有力的作品集。

參考文獻與延伸閱讀

- ISO 13482:2014, Robots and Robotic Devices — Safety Requirements for Personal Care Robots（個人照護機器人的安全標準）。
- ISO 12100:2010, Safety of Machinery — General Principles for Design（風險評估的基礎標準）。
- 經濟部標準檢驗局（BSMI）商品檢驗相關規定與國家標準（CNS）：台灣的商品檢驗要求。
- 國家通訊傳播委員會（NCC），電信管制射頻器材型式認證相關規定：無線電設備的認證要求。
- 《個人資料保護法》及其施行細則（中華民國）：個資保護的法源。

- European Commission, Regulation (EU) 2023/1230 on Machinery (歐盟機械法規，2027 年起取代機械指令) 。
- Moore, G. A., Crossing the Chasm, HarperBusiness (市場採用曲線與鴻溝理論的經典著作) 。
- Rogers, E. M., Diffusion of Innovations, 5th ed., Free Press (創新擴散理論的原始著作) 。
- Mori, M., "The Uncanny Valley," IEEE Robotics & Automation Magazine, 2012 (英譯版；恐怖谷概念的原始論文) 。
- Bartneck, C. et al., Human-Robot Interaction: An Introduction, Cambridge University Press (人機互動的完整教材) 。
- Kanda, T. and Ishiguro, H., Human-Robot Interaction in Social Robotics, CRC Press (社交機器人的互動設計) 。
- International Federation of Robotics (IFR), World Robotics: Service Robots (年度出版)：服務機器人的市場資料與趨勢。

第 36 章

總整專題方法論

編著：李世淵 (機器人叫獸)

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩組專題的對照：上方是一台掛滿功能標籤 (導航、辨識、抓取、語音、陪聊) 的機器人，每個標籤旁都標著低落的數字 (60%、40%、55%.....)，旁註「五件事的 60%」；下方是一台只做一件事的小車正精準對接充電座，旁附一張完整的測試表格 (五種地板 × 三種光線 × 六個角度，成功率 98.7%)，旁註「一件事的 95%」。右半為專題的推進路徑圖：由左至右五個階段 (選題界定、規劃、最小可行系統、測試量化、失敗分析與改良)，其中「最小可行系統」以一條貫穿所有層級的垂直細線標示 (對比另一條逐層堆疊的水平線並打叉)，並在「失敗分析」處畫出五個為什麼的追問鏈。下緣文字：專題的價值，不在於你嘗試了多少，而在於你證明了什麼。

叫獸開講 「幫我把桌上的杯子拿到廚房」——這句話難在哪裡？

專題發表會的前一週，兩組學生來找叫獸。

第一組做了一台「全自動智慧居家服務機器人」。他們的規劃是：能導航、能辨識物體、能抓取、能語音對話、還能陪聊天。做了八個月，現在的狀況是——導航會撞牆、辨識只認得三種物體、手臂抓不穩、語音常誤判。「**每一項都做了，但每一項都不能用。**」

第二組只做了一件事：一台會自己找到充電座並精準對接的小車。功能簡單到有人笑他們「這也算專題？」。但他們做了完整的測試——在五種地板材質、三種光線條件、六個不同的起始角度下，各測 50 次，成功率 98.7%，並記錄了失敗的 13 次分別是什麼原因。

叫獸看完後說：「**第二組的分數會比較高。**」

第一組不服氣：「可是我們做的功能多那麼多！」

「我問你們一個問題。」叫獸說，「**你們的機器人，能不能穩定地做完一件事？**」

沉默。

「這就是差別。」叫獸在白板上寫下：

做到一件事的 95%，勝過做到五件事的 60%。

「而且原因不只是分數。」他繼續說，「**第二組學到的東西比你們多。**他們知道為什麼在磁磚上會失敗、知道光線角度如何影響對接、知道 13 次失敗中有 9 次是同一個原因。**這些是真正的知識。**」

「而第一組呢——你們花了八個月，最後只知道『每一項都很難』。**這不是知識，這只是感想。**」

小翰低聲說：「那我們現在該怎麼辦？」

「還有一週。」叫獸拿起筆，「**選一個功能，把它做到能穩定運作，並且量化它。**剩下的功能就誠實地說『這是未來的工作』——**這比假裝都做到了誠實得多，也專業得多。**」

最後他補了一句，那句話成為這一章的核心：

「一個專題的價值，不在於你嘗試了多少，而在於你證明了什麼。」

本章為什麼重要

這是全書的最後一章。前面三十五章教的是**機器人學是什麼**，本章要教的是——**如何用它做出你自己的東西**。

專題 (capstone project) 是多數學生第一次獨立完成一個完整的工程作品。它與課堂作業有本質的不同：

- **沒有標準答案**：課本的習題有解答，專題沒有。你必須自己判斷什麼是對的。
- **資源有限**：時間、經費、人力都受限。**必須做取捨**——而取捨的能力，正是工程師與學生的差別。
- **會失敗**：而且失敗是常態。**如何從失敗中提取知識**，決定了這個專題對你的價值。
- **需要溝通**：做出來只是一半，讓別人理解你做了什麼、為什麼有價值，是另一半。

本章的目標不是教你「**怎麼做一個好專題**」，而是教你「**怎麼判斷什麼是好專題**」——因為前者依題目而異，後者卻是通用的。

而本章也是全書所有內容的收束：第 34 章的系統工程、第 35 章的產品化思維、第 30 章的漸進式驗證、第 33 章的倫理檢核——它們都將在你的專題中被實際使用。

學習目標

- 能說明專題的價值來源並區分「嘗試」與「證明」。
- 能運用三個判準評估一個題目是否適合。
- 能界定專題的範圍並辨識過度擴張的徵兆。
- 能撰寫可驗證的專題目標與成功判準。
- 能運用倒推法規劃時程並辨識關鍵路徑。
- 能設計最小可行系統並規劃迭代的順序。
- 能建立有效的實驗記錄與版本管理習慣。
- 能設計量化的測試方案並統計地呈現結果。
- 能進行系統性的失敗原因分析。
- 能依失敗分析提出並驗證設計改良。

- 能撰寫結構清晰的專題報告與有效的簡報。
- 能誠實地評估專題的貢獻與限制。

先備知識

本章需要的是全書的整體理解。特別會用到：第 34 章的系統工程與需求管理、第 30 章的漸進式驗證、第 29 章的軟體工程實務、第 35 章的社會與倫理考量、以及第 2 章的學習方法。

建議的讀法：若你正要開始專題，現在就讀第 36.2~36.5 節（選題與規劃）；若你已在進行中，讀 36.6~36.8 節（開發與測試）；若接近尾聲，讀 36.9~36.11 節（分析與發表）。這一章是可以「按需查閱」的。

核心名詞中英對照

表 36-1 本章核心名詞

中文	英文	說明
總整專題	Capstone Project	整合所學的綜合性專題
範圍界定	Scoping	明確定義做什麼與不做什麼
最小可行系統	Minimum Viable System	能端到端運作的最簡版本
垂直切片	Vertical Slice	貫穿所有層級的最小功能
關鍵路徑	Critical Path	決定專案總時程的工作序列
緩衝時間	Buffer / Slack	為不確定性預留的時間
里程碑	Milestone	可驗證的階段性成果
消融實驗	Ablation Study	移除某元件以驗證其貢獻
基準線	Baseline	用於比較的參考方法
對照組	Control Group	實驗中未施加處理的比較組
可重現性	Reproducibility	他人能重複得到相同結果
失效模式分析	Failure Mode Analysis	系統性地分析失敗的原因
根本原因	Root Cause	問題的最終來源而非表象
五個為什麼	Five Whys	反覆追問以找出根本原因
設計改良	Design Iteration	依分析結果修改設計
技術債	Technical Debt	為求快速而累積的品質欠缺
同儕評閱	Peer Review	由同領域者審查工作

中文	英文	說明
電梯簡報	Elevator Pitch	極短時間內說明核心價值

36.1 專題的價值從哪裡來

36.1.1 嘗試與證明的差別

開頭故事的核心，是一個常被誤解的問題：**專題的價值是什麼？**

多數學生以為是「**做了多少功能**」。但實際上，評審（以及未來的雇主）看的是：

你證明了什麼？
你怎麼知道你證明了它？
別人能相信你的結論嗎？

這三個問題，構成了專題價值的全部。

第一組的問題不在於「功能太多」，而在於「什麼都沒證明」。他們無法回答「你的導航有多準？」「辨識率是多少？」「在什麼條件下會失敗？」——因為他們從未量化過。

第二組雖然只做一件事，卻能明確地說：「在五種材質、三種光線、六個角度下各測 50 次，成功率 98.7%，失敗的 13 次中有 9 次源於同一個原因。」——這是一個可以被檢驗、被引用、被延續的結論。

36.1.2 專題的四種價值

一個專題可以在四個面向創造價值，而多數專題只需要在其中一項做得好：

表 36-2 專題的四種價值來源

價值	問題	評判標準	例子
新穎性	「這件事以前沒人做過」	文獻中確實沒有	新的演算法、新的機構
性能	「我做得比現有的好」	與基準線的量化比較	成功率提升 15%
整合	「我把這些技術串起來了」	系統確實能端到端運作	完整的服務機器人原型
理解	「我搞清楚了為什麼」	分析的深度與可信度	失敗原因的系統性分析

大學部的專題，最容易創造價值的是第三與第四項——整合與理解。而第四項「理解」尤其被低估：一個誠實地分析「為什麼我的方法在這種情況下會失敗」的專題，其價值往往高於一

個宣稱「全部成功」卻沒有細節的專題。

◎ 為何「失敗的分析」是有價值的貢獻

在學術與工程領域，一個被充分理解的失敗，比一個沒被理解的成功更有價值。原因是：成功可能來自運氣或未被察覺的特殊條件（第 30 章的 Sim-to-Real 落差就是最好的例子）；而一個被分析清楚的失敗，告訴了後人「這條路的邊界在哪裡」。本書多處引用的重要論文中，有不少正是「指出某方法的失效條件」的工作。因此若你的專題結果不如預期，不要隱藏它——要把它變成你的主要貢獻。這需要誠實與勇氣，但那正是專業的表現。

36.1.3 三個常見的誤區

誤區一：功能越多越好（開頭故事的第一組）。實際上，**深度勝過廣度**——做到一件事的 95%，勝過五件事的 60%。

誤區二：用最新的技術就是好。使用大型模型不會自動使專題變好；若一個簡單的方法就能解決，用複雜的方法反而顯示判斷力不足（呼應第 31.1.2 節的判準）。

誤區三：等做完了再測試。這是最致命的一項——若最後一週才發現不行，就沒有補救的機會了（36.5 節會詳述）。

36.2 選題

36.2.1 三個判準

一個好的專題題目，必須同時滿足三個條件。**缺一項就會出問題**：

表 36-3 選題的三個判準

判準	問題	不滿足的後果
有價值	做出來之後，誰會在意？	做完了但「所以呢？」
做得完	在你的時間與資源內能完成嗎？	最常見的失敗原因
你能學到東西	這個題目能讓你成長嗎？	做完了但沒有收穫

第二項是最常被低估的。學生往往高估自己在單位時間內能完成的量——這是所謂的「**規劃謬誤**」（planning fallacy），是一個普遍的認知偏誤。

一個實用的校準方法：估算你認為需要的時間，然後乘以 2.5。這個係數聽起來誇張，但符合多數專案的實際經驗。

36.2.2 題目的三種來源

來源一：真實的痛點（最推薦）。觀察你周遭的重複性工作、不便之處、或危險的作業。

- **優點：**需求真實、容易說明價值、且**你能實際驗證**。
- **如何找：**用第 34.2.2 節的五問法訪談實際的使用者。**去現場觀察至少三次。**

來源二：延續既有的工作。接續學長姊的專題、實驗室的研究方向、或開源專案。

- **優點：**有基礎可以站，不必從零開始；且有人能指導。
- **風險：**可能繼承了難以理解的程式碼與未記錄的坑。**接手前務必先確認能不能跑起來。**

來源三：複現與延伸。先複現一篇論文或一個開源專案，再做一個小的改進。

- **優點：**目標明確、有基準線可比較（這對第 36.2 節的「性能」價值極為重要）。
- **注意：**複現本身就有價值——許多論文的結果難以複現，若你能誠實地報告「我複現的結果與論文有差距，原因是……」，這就是貢獻。

36.2.3 題目的健康檢查

在確定題目前，用以下六個問題自我檢查：

1. **能不能用一句話說清楚？**若需要三分鐘才能解釋，題目可能太模糊或太大。
2. **成功長什麼樣子？**若說不出「做到什麼程度算成功」，就無法規劃也無法驗收。
3. **最難的部分是什麼？**若答不出來，代表你還沒真正理解這個題目。
4. **如果那個最難的部分做不出來，還剩下什麼？**——這是最重要的一個問題（36.3.3 節）。
5. **現有的方法為什麼不夠？**若現成的工具就能解決，那你的貢獻是什麼？
6. **你能取得所需的資源嗎？**硬體、資料、場地、以及能指導你的人。

◎ 一個危險的題目類型：「等硬體到就開始」

有一類題目特別危險：**整個專題都建立在某個尚未到手的硬體上**。「等那台機器手臂到貨我就開始寫程式」「等感測器買回來就能測試」——**這在實務上幾乎必定延誤**。採購可能延遲、規格可能不符、到貨後可能發現不能用。**因應之道有三：**一、**先用模擬開發**（第 30 章的價值在此體現）；二、**先用手邊的替代品**（如用手機相機代替工業相機做初步驗證）；三、**把硬體相依的部分隔離**，使其他部分能並行推進。**最重要的原則是：不要讓任何單一的外部依賴，成為整個專題的必要條件。**

36.3 範圍界定

36.3.1 範圍是專題成敗的關鍵

第 34.2.4 節提過「範圍潛變」是專案殺手。在學生專題中，這個問題**更加嚴重**——因為沒有客戶會限制你，只有你自己會不斷加東西。

過度擴張的四個徵兆：

- 「順便加一個……」：每次想到新功能就加進來。
- 「這個做完再說」：核心功能還沒完成就開始做次要功能。
- 「應該不難」：對未做過的事情估計過於樂觀。
- 無法說出「不做什麼」：若你列不出明確的排除清單，代表範圍還沒界定。

36.3.2 寫下「不做什麼」

這是界定範圍最有效的一招（呼應第 34.2.4 節）。一份好的範圍文件應該有兩欄：

表 36-4 範圍界定的範例（自動對接充電的小車）

本專題包含（✓）	本專題不包含（X）
在已知地圖中導航到充電站附近	未知環境的自主探索
用視覺標記做精準對接（誤差 < 5 mm）	無標記的自然特徵對接
在平坦室內地面運作	斜坡、門檻、戶外
電量低於 20% 時自動返回	電池管理與充電曲線最佳化
對接失敗時重試三次並回報	多台機器人的充電排程
量化測試五種地板、三種光線	溫度、濕度等環境變因

右欄不是「做不到」，而是「這次不做」。明確地寫出來有三個好處：

- 保護自己：避免被要求「順便做一下」。
- 聚焦資源：把有限的時間投在核心功能上。
- 誠實地呈現：報告中寫「本研究不涵蓋 X」，比被人問到才說「呃我們沒做」專業得多。

36.3.3 最重要的問題：如果最難的部分失敗了呢

這是 36.2.3 節第四個問題的展開，也是專題規劃中最關鍵的一步。

每個專題都有一個「若失敗則全盤皆輸」的環節。你必須提前識別它，並準備退路：

表 36-5 關鍵風險與退路的規劃範例

關鍵風險	若失敗	退路 (Plan B)
視覺對接的精度不足	無法自動充電	改用機械導引槽 (第 33.3 節的形態計算)
深度學習模型訓練不起來	辨識失效	改用傳統的特徵比對 (第 22.5 節)
實體機器人採購延遲	無法測試	先在模擬中完成 (第 30 章)
SLAM 在該場地漂移嚴重	導航失準	改用視覺標記定位 (第 22.4.4 節)

注意每一個退路都指向本書中「更簡單但更可靠」的方法。這不是妥協，而是工程判斷——一個能運作的簡單方案，勝過一個做不出來的先進方案。

◎ 退路不是失敗，而是專業

許多學生認為「準備 Plan B 就是對自己沒信心」。恰恰相反——沒有 Plan B 才是不專業。工程的本質就是在不確定性中推進，而識別風險並準備因應，正是工程師的核心能力 (第 34.4.2 節的 POC 也是同樣的思維)。更重要的是：有了退路，你才敢大膽地嘗試困難的方法——因為你知道即使失敗也不會全盤皆輸。這反而使你更可能成功。

36.4 規劃

36.4.1 倒推法

規劃時程最有效的方法是由截止日往回推，而非由今天往前排。

具體的步驟：

1. 寫下最終的截止日 (發表日)。
2. 往前推兩週：這是「必須完全停止開發」的日期——剩下的時間用於整理數據、撰寫報告、製作簡報、排練。這兩週不可壓縮。
3. 再往前推兩週：這是「必須完成所有測試」的日期。若測試發現問題，這兩週是修補的緩衝。
4. 剩下的時間才是開發。
5. 在開發時間中扣除 30% 作為緩衝 (意外、生病、其他課業、硬體延遲)。

一個常見的震撼：以 16 週的專題為例：

總時程 16 週

$$\begin{aligned}
 & - \text{報告與簡報 } 2 \text{ 週} - \text{測試與修補 } 2 \text{ 週} = 12 \text{ 週} \\
 & \times (1 - 30\% \text{ 緩衝}) = \text{實際可用的開發時間僅 } 8.4 \text{ 週}
 \end{aligned}$$

這通常只有學生原本預期的一半。

這個計算的價值在於：它會逼你重新界定範圍（36.3 節）。與其在最後兩週手忙腳亂，不如一開始就把範圍縮到 8.4 週能完成的程度。

36.4.2 里程碑的設計

好的里程碑必須是**可驗證的**——能明確判斷「達成了」或「還沒」。

表 36-6 里程碑的兩種寫法

模糊的（不可驗證）	明確的（可驗證）
「完成視覺模組」	能在測試影像上輸出標記的位姿，誤差 < 10 mm
「機器人會動」	能接收速度指令並在地面移動，誤差 < 5%
「整合完成」	能端到端執行一次完整流程（即使成功率低）
「開始測試」	完成 50 次測試並記錄成功率

第三列特別重要——「端到端執行一次」應該是一個很早的里程碑（36.5 節）。

36.4.3 辨識關鍵路徑

關鍵路徑是決定總時程的那條工作序列——它上面的任何延誤都會直接延誤整個專題。

辨識的方法：畫出各項工作的相依關係，找出最長的那條鏈。

例：硬體採購（3 週）→ 組裝（1 週）→ 校正（1 週）→ 整合測試（2 週）= 7 週
而軟體開發可以並行（5 週）

關鍵路徑是硬體那一條——因此應優先啟動採購，而非先寫程式。

三個實務建議：

- 最長前置期的項目最先啟動：硬體採購、需要申請的場地、需要協調的受試者。
- 關鍵路徑上的工作要留最多緩衝。
- 盡可能把工作並行化：例如用模擬先開發軟體，不等硬體到貨（36.2.3 節的知識框）。

36.4.4 團隊的分工

多數專題是團隊進行的，而分工的方式會深刻影響成敗。

兩種常見的分工模式：

表 36-7 兩種分工模式的比較

模式	作法	優點	風險
依模組分工	A 做視覺、B 做控制、C 做機構	各自專精、效率高	整合時才發現接不起來（第 34.3.2 節）
依垂直切片分工	每人負責一個完整的功能流程	整合風險低、每人都懂全貌	需要更多的溝通與共同學習

實務建議是「混合」：初期用垂直切片讓每個人都跑通一次端到端的流程（建立共同的理解），之後再依模組分工深化。

三個必要的團隊機制：

- 共用的版本管理（Git）：這不是選配。沒有版本管理的團隊專題，必定會發生「檔案覆蓋」與「不知道誰改了什麼」的災難。
- 定期的同步：每週一次固定的會議，各自報告進度、卡關、與下週計畫。卡關要早說——這是團隊最大的價值。
- 明確的介面約定：即使依模組分工，也要先約定好模組之間的介面（第 34.3.2 節的介面規格書）。

36.5 最小可行系統

36.5.1 先讓它跑起來，再讓它跑得好

這是本章最重要的一個原則，也是最常被違反的一個。

◎ 垂直切片優於水平堆疊

錯誤的作法（水平）：先把視覺模組做到完美、再把控制模組做到完美、最後整合。結果是：直到最後一個月才第一次整合，然後發現接不起來。正確的作法（垂直）：先做一個粗糙但能端到端運作的版本——視覺只認得一個固定的標記、控制只會直線移動、抓取只會固定姿態——但它能從頭走到尾。之後再逐項深化。這個「粗糙的完整版本」稱為最小可行系統（Minimum Viable System），它應該在專題的前三分之一就完成。

為何垂直切片如此重要？三個理由：

- 及早暴露整合問題（第 34.3.2 節）：時序、座標系、資料格式的問題會立刻浮現，而非在最後一週。
- 建立可運作的基準：之後每一次改進都能與它比較，知道自己是否真的變好了。
- 降低心理壓力：有一個「至少能動」的版本在手，你才敢大膽地嘗試困難的改進。

36.5.2 如何設計最小可行系統

以「自動對接充電」的專題為例：

表 36-8 最小可行系統與完整系統的對照

環節	最小可行版本	完整版本
定位	手動放在起點附近	自主導航到充電站附近
偵測	只認一個大尺寸的固定標記	多尺度、抗遮蔽的標記偵測
對位	開迴路移動固定距離	視覺伺服的閉環對位
判定	人工判斷是否對接成功	電流偵測自動判定
異常	失敗就停止	重試策略與回報

這個最小版本可能一週就能完成——而它一旦運作，你就知道整個流程是通的，接下來只需逐項替換為完整版本。

深化的順序原則：先做風險最高的（36.3.3 節的關鍵風險），而非最容易的。若視覺對位是關鍵風險，就先深化它。

36.5.3 用模擬加速迭代

第 30 章的模擬在專題中有三個具體的價值：

- 不等硬體：軟體開發可以在硬體到貨前開始（36.2.3 節的知識框）。
- 快速試錯：一個參數的效果，在模擬中十分鐘就能測完，在實機上可能要半天。
- 安全地測極端情況：故意讓機器人撞牆、故意讓感測器失效——這些在實機上代價太高。

但也要注意第 30 章的警告：模擬中成功不代表實機成功。因此最小可行系統應該盡早在實機上跑通一次，即使很粗糙。

36.6 開發的實務

36.6.1 版本管理：不是選配

任何專題都應該使用 **Git**（或其他版本管理）。三個理由：

- 能回到任何一個版本：當你改壞了，能立刻退回昨天能跑的版本。這救過無數個專題。
- 能記錄「為什麼改」：good commit message 是最好的實驗紀錄。
- 團隊協作的唯一方式：沒有 Git 的團隊專題，必定會發生檔案覆蓋的災難。

三個實用的習慣：

- 每次「能跑」就 **commit**，並在訊息中寫清楚改了什麼、為什麼。
- 用 **tag** 標記重要的版本：`v1-milestone-first-e2e`（第一次端到端成功）——之後要做比較或回退時極為方便。
- 參數與程式碼分離（第 29.5.1 節）：把可調的參數放在設定檔中，這樣調參時不會弄髒程式的歷史。

36.6.2 實驗紀錄

這是專題中最容易被忽略、但價值最高的習慣。

為什麼需要：三個月後你會完全忘記「當時為什麼把那個參數設成 0.3」。而寫報告時，你會需要那些數據。

一份好的實驗紀錄應該包含：

```
## 2026-03-15 視覺對位測試 #7

### 目的
測試不同光照下的標記偵測穩定度

### 設定
- 程式版本：commit a3f9c21
- 參數：threshold=0.65, min_marker_size=40px
- 環境：實驗室，日光燈全開 / 半開 / 關閉（僅窗光）
- 標記尺寸：10 cm x 10 cm

### 結果
| 光照條件 | 測試次數 | 偵測成功 | 平均誤差 |
```

-----	-----	-----	-----
全開	30	30	3.2 mm
半開	30	28	4.1 mm
僅窗光	30	19	8.7 mm

觀察

- 光線不足時偵測率明顯下降 (63%)
- 失敗的 11 次中，有 8 次是標記邊緣模糊導致角點偵測失敗
- 誤差在低光下也變大，推測與角點精度有關

下一步

- 試著把 threshold 降到 0.5，看能否改善低光的偵測率
- 考慮加裝補光 (但要注意反光，見 #5 的教訓)

注意這份紀錄的四個要素：目的、設定 (含程式版本)、量化的結果、以及觀察與下一步。

最後一項使紀錄不只是數據，而是思考的軌跡。

◎ 實驗紀錄的複利效應

寫實驗紀錄看起來是額外的工作，實際上它會省下更多時間。三個具體的回報：一、避免重複的錯誤——「這個我上次試過了，不行，原因是……」；二、寫報告時直接可用——你不需要回想，數據和分析都在那裡；三、討論時的依據——當指導老師問「你怎麼知道？」，你能立刻拿出證據。一個實務的建議：每次做完實驗立刻寫，不要拖到明天。因為細節會在幾小時內就模糊掉。

36.6.3 除錯的方法

除錯佔了開發時間的大半。四個有效的原則：

原則一：二分法。當一個長流程出錯時，先確認**中間點**是對是錯，就能把問題範圍減半。反覆數次即可定位。

原則二：先驗證假設，而非猜測。與其猜「應該是視覺的問題」，不如印出視覺的輸出來看。第 29.9.2 節的 RViz 與第 29.9.1 節的 `ros2 topic echo` 正是為此。

原則三：一次只改一件事。同時改三個地方然後「好像好了」，你不會知道是哪一個起了作用——而下次遇到類似問題時也無從累積經驗。

原則四：能重現才能修。若一個問題偶爾才發生，第一件事是找出讓它穩定重現的條件。無法重現的問題無法確認是否修好。

36.6.4 技術債的管理

技術債是指為了求快而暫時採用的「不好但能動」的作法。在專題中，適度地欠債是合理的——但必須知道自己欠了什麼。

實用的作法：在程式中用統一的標記記下：

```
# TODO: 這裡的座標轉換寫死了，應該改用 TF ( 第 29.6 節 )
#       目前能動是因為相機位置固定。若相機會動就會壞。

# HACK: 用 sleep(0.5) 等待視覺結果，應該改用回呼
#       這會使節拍變慢，但先求能動

# FIXME: 這個閾值 0.65 是試出來的，不知道為什麼
#       在低光下會失效 ( 見實驗紀錄 #7 )
```

三種標記的區別：`TODO` 是「該做但還沒做」、`HACK` 是「能動但不對」、`FIXME` 是「已知有問題」。在報告的「限制」一節中，這些標記就是現成的素材。

36.7 測試與量化

36.7.1 從「能動」到「有多好」

開頭故事的核心差異就在這裡。「我的機器人能自動對接」是一個沒有資訊的陳述——它可能是十次成功一次，也可能是百次成功九十九次。

量化的三個要素：

在什麼條件下 (測試的情境)
測了幾次 (樣本數)
結果如何 (成功率、誤差、時間)

缺任何一項，結論都無法被檢驗。

36.7.2 測試方案的設計

一份好的測試方案應該涵蓋四類情境：

表 36-9 測試情境的四個類別

類別	目的	例子 (自動對接)
標準情境	驗證基本功能	正常光線、平坦地面、正對充電站

類別	目的	例子 (自動對接)
變異情境	測試強健性	不同光線、不同地板、不同起始角度
邊界情境	找出能力的極限	最遠可以從多遠開始對接？最大偏角？
異常情境	驗證失效的行為	標記被部分遮蔽、途中有障礙、電量極低

第三類「邊界情境」最能展現專業度。一個能明確說出「本系統在起始偏角 $\pm 30^\circ$ 內成功率 $> 95\%$ ，超過 40° 後急遽下降」的專題，遠勝於只說「我們的系統很穩定」。

36.7.3 樣本數要多少

這是學生最常問的問題。一個實用的判準：

要區分 90% 與 95% 的成功率，每組至少需 30~50 次
要宣稱「 $> 99\%$ 」，至少需要 300 次以上

因為 30 次全成功，只能說明成功率大約在 88% 以上 (95% 信心)。

一個常見的錯誤：測了 10 次全成功就宣稱「成功率 100%」。統計上，10 次全成功只能支持「成功率大於 74%」的結論 (95% 信心區間的下界)。

實務建議：每個測試情境至少 30 次；若要做情境之間的比較 (如「改良前 vs 改良後」)，每組至少 50 次。

36.7.4 基準線與消融實驗

基準線 (baseline) 是用來比較的參考。沒有基準線的性能數字沒有意義——「成功率 92%」是好是壞？要看比什麼。

三種常見的基準線：

- 現有的方法：論文中的方法、開源的實作、或商用產品。
- 簡單的方法：一個「笨」但能動的作法。若你的複雜方法只比簡單方法好一點點，這本身就是重要的發現。
- 你自己的前一版：改良前後的比較。

消融實驗 (ablation study) 則是驗證「每個元件是否真的有貢獻」的方法：逐一移除某個元件，看性能下降多少。

表 36-10 消融實驗的範例

配置	成功率	平均誤差	結論
完整系統	98.7%	3.2 mm	—
移除視覺伺服（改開迴路）	62.3%	18.4 mm	視覺伺服貢獻極大
移除補光	84.0%	4.1 mm	補光在低光時重要
移除重試機制	94.1%	3.2 mm	貢獻有限（僅救回 4.6%）
移除濾波	97.9%	3.5 mm	幾乎沒有貢獻

最後一列的發現極有價值——它顯示「濾波」這個元件幾乎沒有作用。誠實地報告這件事，比假裝每個元件都很重要更專業（而且可以在未來版本中移除它以簡化系統）。

36.7.5 結果的呈現

三個原則：

- 用表格呈現數字，用圖呈現趨勢。若要比較數個離散的配置，用表格；若要顯示某個參數的影響，用曲線圖。
- 標示變異性：不要只報平均值，要報標準差或範圍。「成功率 92%」與「92% ± 3%」是不同的資訊量。
- 誠實地呈現失敗：失敗的案例應該被分析而非隱藏（36.8 節）。

36.8 失敗原因的分析

36.8.1 這是專題最有價值的部分

如 36.1.2 節的知識框所述，一個被充分理解的失敗，比一個沒被理解的成功更有價值。而失敗分析正是把「失敗」轉化為「知識」的程序。

多數學生的處理方式是：「我們的系統成功率 85%，失敗的原因是環境干擾。」——這幾乎沒有資訊。「環境干擾」是什麼？哪一種干擾？佔多少比例？

專業的處理方式是：對每一次失敗逐一記錄與分類。

36.8.2 失敗的分類

以開頭故事第二組的 13 次失敗為例：

表 36-11 失敗案例的分類與分析

失敗模式	次數	佔比	初步原因	根本原因
標記偵測失敗	9	69%	光線不足	曝光時間固定，未依環境調整
對位誤差過大	2	15%	地板打滑	輪速回授失準（第 23.8 節）
通訊逾時	1	8%	WiFi 不穩	未設重連機制
未知	1	8%	無法重現	—

這張表的價值在於三點：

- 它指出了主要矛盾：69% 的失敗來自同一個原因——因此改善的資源應該集中在那裡（呼應第 34.5.2 節「找出瓶頸」）。
- 它區分了初步原因與根本原因：「光線不足」是現象，「曝光固定」才是可以修改的原因。
- 它誠實地保留了「未知」：不強行歸類無法解釋的案例。

36.8.3 五個為什麼

要從現象追到根本原因，最實用的方法是**五個為什麼**（Five Whys）——與第 34.2.2 節的五問法同源，但用於故障分析。

◎ 五個為什麼的實例

現象：對接失敗。**為什麼？**——因為機器人停在偏離充電座 2 公分的地方。**為什麼？**——因為視覺算出的位置有誤差。**為什麼？**——因為標記的角點偵測不準。**為什麼？**——因為影像模糊。**為什麼？**——因為曝光時間固定為 20 ms，在低光環境下不足，導致影像雜訊高且對比不足。**根本原因：**曝光參數未依環境自適應。——注意這個結論是可以被修改的，而最初的「對接失敗」不是。這就是根本原因分析的價值：把「不知道怎麼辦」轉化為「知道要改哪裡」。

36.8.4 由分析到改良

找出根本原因後，改良的方案應該**針對根本原因**，並且**再次量化驗證**。

完整的改良循環有四步：

1. **提出改良方案：**針對根本原因。以上例而言：改用自動曝光、或加裝補光。
2. **預測效果：**在改之前先預測——「若曝光是主因，改善後低光下的偵測率應由 63% 提升到 90% 以上」。這一步極為重要（見下方知識框）。
3. **實作並重測：**用相同的測試條件重測，才能比較。

4. 比對預測與實際：若符合預測，代表你的因果分析正確；若不符合，代表根本原因找錯了一—回到第一步。

◎ 為何要「先預測再驗證」

這一步是區分「工程」與「試誤」的關鍵。若你改了一個東西然後變好了，這不代表你理解了原因——可能是巧合，也可能是別的因素。但若你先預測「改了之後應該會從 63% 提升到 90%」，然後實際測出 91%，這就強烈地支持了你的因果理解。反之，若你預測 90% 卻只得到 68%，你就知道「還有別的原因沒找到」——這比盲目地繼續試更有價值。這個習慣會讓你的專題報告從「我們試了很多方法」升級為「我們理解了這個系統」。

36.8.5 改良的成果呈現

改良的成果應該用改良前後的對照呈現：

表 36-12 改良前後的對照 (範例)

測試條件	改良前	改良後	提升
全開日光燈	100% (30/30)	100% (30/30)	—
半開	93.3% (28/30)	100% (30/30)	+6.7%
僅窗光 (低光)	63.3% (19/30)	93.3% (28/30)	+30.0%
整體	85.6%	97.8%	+12.2%

注意這張表的說服力來源：它顯示改良只在預期的情境 (低光) 產生顯著效果——這正好驗證了「根本原因是曝光」的分析。若改良在所有情境都提升了，反而值得懷疑是否有其他因素。

36.9 報告的撰寫

36.9.1 報告的結構

一份技術報告的標準結構，其實反映了一個論證的邏輯：

表 36-13 報告的結構與其論證功能

章節	回答的問題	常見的錯誤
摘要	整篇的濃縮 (問題、方法、結果、結論)	寫成「背景介紹」而非結果的摘要
緒論	為什麼這個問題重要？	只講背景不講動機
文獻回顧	別人做到哪裡？還缺什麼？	只是列舉而未指出「缺口」
方法	你做了什麼？別人能複現嗎？	細節不足、參數未交代

章節	回答的問題	常見的錯誤
實驗與結果	你怎麼知道它有效？	只有「能動」的展示而無量化
討論	這些結果代表什麼？限制是什麼？	沒有討論限制
結論	所以呢？下一步是什麼？	只是重複摘要

「實驗與結果」是報告的核心，也是最常被寫得最薄弱的一節。多數學生的報告在「方法」寫了十頁，「結果」只有兩頁——這個比例應該倒過來。

36.9.2 摘要的寫法

摘要是被讀最多次的部分（許多人只讀摘要）。它應該包含四個要素，各一到兩句：

【問題】室內移動機器人的自動充電對接，在低光環境下的可靠度不足。

【方法】本研究提出一套結合視覺標記偵測與自適應曝光的對接系統，並以視覺伺服進行閉環對位。

【結果】在五種地板材質、三種光照條件下各測試 50 次，整體成功率由基準版本的 85.6% 提升至 97.8%；低光環境的改善尤其顯著（63.3% → 93.3%）。失敗案例的分析顯示，剩餘的失敗中有六成源於地板打滑造成的里程計誤差。

【結論】自適應曝光是低光對接的關鍵；未來的改善應聚焦於里程計的強健性而非視覺模組。

注意這份摘要的特徵：有具體的數字、有與基準的比較、有失敗的分析、且結論指出了下一步的方向。這比「本研究成功實現了自動對接功能」有價值一百倍。

36.9.3 誠實地寫「限制」

「限制」一節是專業度的試金石。許多學生擔心寫出限制會扣分——恰恰相反。

理由有三：

- 評審看得出來。你的系統一定有限制，若你不寫，評審會在提問時指出——而那時你就顯得沒有自知之明。
- 主動寫出來反而顯示理解深度。能清楚說出「本系統在何種條件下會失效」，證明你真正理解了它。

- 這是後續研究的價值（呼應第 35.3.2 節「誠實的限制說明反而是優勢」）。

一份好的限制說明的例子：

研究限制

1. 環境限制：本系統僅在平坦室內地面驗證。斜坡、門檻與地毯未經測試，預期在地毯上因里程計誤差增大而性能下降。
2. 標記依賴：本系統依賴事先貼設的視覺標記（第 22.4.4 節）。無標記的自然特徵對接不在本研究範圍內。
3. 樣本數：每個測試情境 50 次，足以區分 90% 與 98% 的成功率，但不足以精確估計 99% 以上的可靠度。
4. 單一機器人：所有測試在同一台機器人上進行，未驗證跨機台的一致性。不同的相機與輪徑可能需要重新校正。
5. 未量化的因素：溫度、濕度、電池電量對性能的影響未納入測試。

注意每一條都具體且可檢驗，而非空泛的「本研究仍有改進空間」。

36.9.4 可重現性

若別人無法複現你的結果，你的結論就無法被檢驗。三個具體的作法：

- 完整交代參數：所有的閾值、增益、尺寸都應該列出（可放附錄）。「經過調整後效果良好」不是可重現的敘述。
- 交代環境：硬體型號、軟體版本、測試場地的條件。
- 公開程式與資料（若可能）：附上 Git 儲存庫的連結。這是現代技術報告的標準作法。

36.10 發表與簡報

36.10.1 簡報不是報告的縮短版

這是最常見的錯誤：把報告的每一節各做一張投影片。

兩者的目的不同：

報告：讓人「完整地理解」（可以慢慢讀、可以回頭）

簡報：讓人「快速地相信」（只有一次機會、無法回頭）

簡報必須有取捨。

36.10.2 簡報的結構

一份 12 分鐘的專題簡報，建議的結構是：

表 36-14 簡報的建議結構（12 分鐘）

段落	時間	內容	關鍵
問題	1.5 分	為什麼這件事值得做	用具體的情境而非抽象的敘述
目標	1 分	你要做到什麼（可驗證的）	明確的數字
方法	3 分	怎麼做的（只講關鍵的設計決策）	不要講每一個細節
展示	2 分	影片：系統實際運作	這是最有說服力的一段
結果	3 分	量化的數據與比較	這是最重要的一段
限制與未來	1 分	誠實的自我評估	顯示專業度
結論	0.5 分	一句話的核心貢獻	讓人記住

注意「結果」佔了最多時間——這正是 36.1.1 節「你證明了什麼」的體現。

36.10.3 影片的重要性

一段 60~90 秒的展示影片，勝過十張投影片。三個製作的建議：

- 一鏡到底，不要剪接。剪接會讓人懷疑「是不是剪掉了失敗的部分」。若一定要剪，明確標示。
- 顯示時間或計數。畫面角落顯示「第 37 次測試」或計時器，增加可信度。
- 也放一段失敗的。這聽起來反直覺，但展示失敗並解釋原因，會大幅提升可信度——它顯示你真的做了大量測試，而非只錄了成功的那一次。

36.10.4 回答問題

問答往往比簡報本身更能顯示你的理解深度。四個原則：

- 先確認問題：若不確定，複述一次「您是問.....對嗎？」。答錯問題比答不出來更糟。
- 誠實地說「不知道」：「這一點我們沒有測試，但我推測.....因為.....」——這比硬掰專業得多。

- 把質疑當成禮物：評審指出的弱點，往往正是你該寫在「未來工作」的內容。
- 準備常見的問題：「為什麼不用 X 方法？」「若換成 Y 環境會怎樣？」「成本多少？」——這三個幾乎必問。

◎ 電梯簡報：三十秒說清楚

練習用三十秒說明你的專題——這個練習的價值極高。若三十秒說不清楚，通常代表你自己還沒想清楚核心是什麼。一個好的三十秒版本的結構是：「很多 [某類使用者] 面臨 [某個問題]，現有的方法 [某個不足]。我們做了 [你的方法]，在 [測試條件] 下達到 [量化結果]，比 [基準] 提升了 [多少]。」——這五句話涵蓋了問題、缺口、方法、驗證與貢獻。這個版本在求職面試、實驗室介紹、與任何非正式的場合都用得上。

36.11 結語：從讀者到創造者

36.11.1 全書的終點

三十六章之後，這本書的內容結束了。

回顧這趟旅程：第一篇我們認識了機器人的身體；第二篇學會描述它能到哪裡；第三篇讓它動得精確而穩定；第四篇讓它理解自己身在何處；第五篇讓它知道該做什麼；而第六篇，我們把這一切變成能在真實世界運作的東西。

而第 33 章留下的那句話，此刻有了新的意義：

現實會回應你，這是機器人學最好的老師。

而本章教的，正是如何聽懂那個回應——透過量化、透過失敗分析、透過誠實的紀錄。

36.11.2 六條主線的最後一次回顧

第 33.10 節整理過全書的六條主線。在你的專題中，它們會全部出現：

表 36-15 六條主線在專題中的體現

主線	在專題中的意義
模型的價值在減負	不完美的模型仍能大幅簡化問題——不要因為模型不準就放棄它
分層與時間尺度分離	把快的與慢的分開處理，是系統能運作的關鍵
設計對不確定性不敏感的系統	與其追求精確，不如追求強健——這往往是專題成敗的分界
機構與控制協同設計	先問能不能用機構解決（36.3.3 節的退路多半屬於此類）
模型提供保證，學習提供泛化	選對工具，而非選最新的工具

主線	在專題中的意義
理論各部分互相支撐	你會發現當初覺得無用的章節，在專題中突然派上用場

36.11.3 最後的話

這本書從第 1 章的一個問題開始：為什麼電腦下棋能贏世界冠軍，卻摺不好一件衣服？

第 33 章給了答案：因為智慧不只在腦裡，也在身體裡，還在身體與環境的互動裡。

但還有一個問題，這本書無法替你回答：

你想做出什麼？

這個問題的答案，只能由你自己給。

三十六章的內容，說到底只是**工具**。運動學、動力學、控制、感知、規劃、學習——它們的價值，取決於你用它們做了什麼。

而做出一個東西的過程，必定包含大量的失敗、挫折、與「為什麼又不行」的深夜。**這是這門學科的代價，也是它的魅力**——因為當它終於動起來的那一刻，那個東西是**真實存在於世界上的**，不是紙上的公式，不是螢幕上的數字。

它會動。而那是你讓它動的。

願你在往後的路上，做出一些讓自己驚訝的東西。

現在，換你了。

完整案例：一個專題的全程

以下用一個虛構但貼近真實的案例，串起本章的所有內容。

案例背景

題目：實驗室移動機器人的自動充電對接系統

時程：16 週 **團隊：**3 人 **資源：**一台差速輪機器人、一支 USB 相機、實驗室場地

第 1~2 週：選題與範圍界定

選題的判斷（36.2.1 節）：

- **有價值 ✓**：實驗室的機器人現在需要人工插電，常因忘記充電而無法使用。
- **做得完 ✓**：範圍明確、硬體現成、且有明確的成功判準。
- **能學到 ✓**：涵蓋視覺、控制、系統整合。

範圍界定（36.3.2 節）：如表 36-4 所示，明確寫出「不含未知環境探索、不含無標記對接、不含斜坡」。

關鍵風險的識別（36.3.3 節）：團隊判斷是「視覺對位的精度能否達到 5 mm」，並準備了退路——若不行，改用機械導引槽（漏斗狀的開口引導）。

第 3~4 週：規劃與最小可行系統

倒推法的時程（36.4.1 節）：

$$16 \text{ 週} - \text{報告簡報} 2 \text{ 週} - \text{測試修補} 2 \text{ 週} = 12 \text{ 週}$$

$$12 \times 0.7 \text{ (緩衝)} = 8.4 \text{ 週的實際開發時間}$$

團隊據此把範圍再縮減——原本想做的「多台排程」被移到「不包含」欄。

最小可行系統（36.5.2 節）：第 4 週結束時，完成了一個「手動放在起點、只認一個大標記、開迴路移動、人工判定」的版本。雖然粗糙，但端到端跑通了——這使團隊立刻發現了兩個沒預料到的問題：相機的視野比預期窄、以及標記在近距離時會超出畫面。

第 5~10 週：逐項深化

深化的順序（36.5.2 節）：先做風險最高的視覺對位。

開發實務（36.6 節）：

- 全程使用 Git，每次能跑就 commit，重要版本打 tag。
- **每次實驗都寫紀錄**（如 36.6.2 節的範例）——這在第 12 週寫報告時省下了大量時間。
- 遇到「偶爾對不準」的問題時，**先找出穩定重現的條件**（36.6.3 節原則四），發現是在特定角度才發生。

第 11~12 週：測試與量化

測試方案（36.7.2 節）：四類情境

表 36-16 案例的測試方案

情境類別	具體條件	次數
標準	日光燈全開、磁磚地、正對	50
變異	五種地板 × 三種光線	$50 \times 15 = 750$ (抽樣至 300)
邊界	起始偏角 $10^\circ \sim 50^\circ$ (每 10° 一組)	$5 \times 30 = 150$
異常	標記部分遮蔽、途中有障礙	60

結果：整體成功率 85.6%；邊界測試發現偏角超過 40° 後成功率由 92% 驟降至 47%——這是一個明確且有價值的發現。

第 13 週：失敗分析與改良

失敗分類 (36.8.2 節)：如表 36-11，發現 69% 的失敗源於低光下的標記偵測。

五個為什麼 (36.8.3 節)：追到根本原因是「曝光時間固定」。

改良與驗證 (36.8.4 節)：

1. 方案：改用自動曝光。
2. 預測：低光下的偵測率應由 63% 提升到 90% 以上；其他光照條件應無顯著變化。
3. 實測：低光 93.3%、其他條件不變。
4. 比對：符合預測 ✓——這強烈支持了因果分析的正確性。

整體成功率由 85.6% 提升至 97.8% (表 36-12)。

第 14~16 週：報告與發表

報告的重點分配 (36.9.1 節)：結果與討論佔了報告的一半篇幅，方法只佔四分之一。

誠實的限制 (36.9.3 節)：明確寫出五項限制，包含「偏角超過 40° 性能急遽下降」——這個「缺點」反而成為簡報時最受好評的部分，因為它顯示團隊真正理解了系統的邊界。

簡報 (36.10 節)：12 分鐘中，結果佔 3 分鐘、展示影片 2 分鐘 (含一段失敗的案例並解釋原因)。

評審的回饋：「你們是唯一一組能明確說出自己系統在什麼條件下會失效的。」

◎ 這個案例的核心教訓

回顧全程，這個專題成功的關鍵不在於技術有多先進——視覺標記偵測是第 22 章的標準技術、視

覺伺服是第 22.7 節的基本方法。它成功是因為五件事：一、範圍界定得夠小（8.4 週能完成）；二、第 4 週就跑通了端到端（及早暴露問題）；三、全程有實驗紀錄（寫報告時省下大量時間）；四、做了充分的量化測試（1,260 次）；五、失敗分析做到了根本原因並驗證了預測。這五件事，任何一個題目都能做到——而它們正是本章的全部內容。

實作：啟動你的專題

本章的實作就是你自己的專題。以下是一份可以直接使用的行動清單。

階段一：選題與界定（第 1~2 週）

1. 列出五個候選題目，用 36.2.1 節的三個判準各自評分（1~5 分）。
2. 對最高分的題目做健康檢查（36.2.3 節的六個問題）。特別是第四個問題——若最難的部分失敗了，還剩下什麼？
3. 寫下範圍文件：兩欄式，明確列出「包含」與「不包含」各至少五項（36.3.2 節）。
4. 識別關鍵風險並準備退路（36.3.3 節），至少列出三個風險與其 Plan B。
5. 寫下可驗證的成功判準：不是「做出一台機器人」，而是「在 X 條件下達到 Y 的成功率」。

階段二：規劃（第 2~3 週）

1. 用倒推法排出時程（36.4.1 節）。算出你實際可用的開發週數——這個數字通常會讓你回頭修改範圍。
2. 設定 4~6 個可驗證的里程碑（36.4.2 節），每個都要能明確判定達成與否。
3. 畫出工作的相依關係圖，標出關鍵路徑（36.4.3 節）。最長前置期的項目今天就啟動。
4. （團隊）約定分工模式與介面規格（36.4.4 節），並建立 Git 儲存庫與每週的同步時間。

階段三：開發（第 3~11 週）

1. 設計最小可行系統（36.5.2 節）：列出每個環節的「最簡版本」。
2. 在前三分之一的時間內完成端到端的跑通——即使很粗糙。
3. 建立實驗紀錄的習慣（36.6.2 節）：每次實驗立刻寫，包含目的、設定、結果、觀察。
4. 每次能跑就 commit，重要版本打 tag。
5. 依風險高低決定深化的順序——先做最可能失敗的。

階段四：測試與分析（第 11~13 週）

1. 設計涵蓋四類情境的測試方案（36.7.2 節）。
2. 每個情境至少測 30 次；若要比較改良前後，每組至少 50 次。
3. 建立基準線：至少要有「簡單方法」或「改良前」作為比較。
4. 逐一記錄每次失敗的現象，不要只記「失敗」。
5. 做失敗分類（36.8.2 節），找出佔比最高的失敗模式。
6. 用五個為什麼追到根本原因（36.8.3 節）。
7. 提出改良、預測效果、實作、比對（36.8.4 節的四步循環）。先預測再驗證。

階段五：發表（第 14~16 週）

1. 先寫摘要（36.9.2 節的四要素）——這會逼你想清楚核心貢獻。
2. 結果與討論應佔報告的一半篇幅。
3. 誠實地寫出至少五項限制（36.9.3 節），每一項都要具體。
4. 附上完整的參數與環境資訊（36.9.4 節的可重現性）。
5. 製作 60~90 秒的展示影片，含一段失敗的案例。
6. 練習三十秒的電梯簡報（36.10.4 節的知識框）。
7. 預想三個必問的問題並準備答案。
8. （最後）用第 33.9.4 節的六個倫理問題檢視你的專題。

國中小也看得懂：不要貪心，選一件事做到最好

假設你要參加科展，有兩個題目可以選：

題目一：「超級無敵萬能機器人」——會走路、會說話、會抓東西、會掃地、會陪你玩。

題目二：「會自己找到插座充電的小車」——只做這一件事。

你會選哪一個？

大部分的人會選第一個，因為聽起來厲害多了。但評審通常會給第二個比較高的分數。為什麼？

因為第一個題目做出來，很可能**每一樣都會，但每一樣都不好用**——走路會撞牆、說話常聽錯、抓東西會掉。

而第二個題目，你可以**把它做到非常好**。而且你可以說出：「我測試了 300 次，成功了 294 次。失敗的 6 次裡，有 4 次是因為地板太滑。」

這就是差別：第一組只能說「我試了很多東西」，第二組可以說「**我證明了一件事**」。

那怎麼做到呢？這一章教了三個祕訣：

祕訣一：先做一個「很爛但完整」的版本。不要先把每個零件做到完美再組起來——**要先讓它從頭到尾動一次**，就算很醜也沒關係。因為這樣你才知道哪裡會出問題。

祕訣二：每次做實驗都要寫下來。寫什麼？做了什麼、結果如何、你發現了什麼。因為三個月後你一定會忘記，而且寫報告的時候會用到。

祕訣三：失敗的時候，要問「為什麼」五次。

比如：對不準 → 為什麼？因為位置算錯 → 為什麼？因為照片模糊 → 為什麼？因為太暗 → 為什麼？因為相機的設定沒有跟著光線調整。

你看，一直問下去，最後就找到「真正可以修的地方」了！

最後告訴你這一章最重要的一句話：

一個專題的價值，不在於你嘗試了多少，而在於你證明了什麼。

所以下次做東西的時候，不要貪心。**選一件事，把它做到最好，然後測試它、記錄它、理解它**。這樣做出來的東西，才真的是你的。

叫獸提醒與常見錯誤

- 誤區一：以為功能越多分數越高。做到一件事的 95%，勝過做到五件事的 60%。
- 誤區二：用最新的技術就以為比較好。若簡單的方法就能解決，用複雜的方法反而顯示判斷力不足。
- 誤區三：先把每個模組做到完美再整合。應先做垂直切片，在前三分之一就跑通端到端。
- 誤區四：低估所需的時間。應把估算乘以 2.5，並用倒推法算出實際可用的開發週數。

- 誤區五：整個專題建立在尚未到手的硬體上。應先用模擬或替代品開發，並隔離硬體相依的部分。
- 誤區六：說不出「不做什麼」。無法列出排除清單，代表範圍還沒界定。
- 誤區七：沒有準備 Plan B。識別關鍵風險並準備退路是專業，不是沒信心。
- 誤區八：不寫實驗紀錄。三個月後你會忘記所有細節，而寫報告時會需要那些數據。
- 誤區九：測了十次全成功就宣稱 100%。統計上這只能支持「大於 74%」的結論。
- 誤區十：沒有基準線就報告性能數字。「成功率 92%」是好是壞，取決於比什麼。
- 誤區十一：把失敗歸因為「環境干擾」就結束。應逐一分類並用五個為什麼追到根本原因。
- 誤區十二：報告中不寫限制。評審看得出來，而主動寫出反而顯示你真正理解了系統。

課堂討論

1. 開頭故事中，叫獸說第二組「學到的東西比較多」。請討論：為何深度勝過廣度？這個原則在什麼情況下不成立？
2. 36.8.4 節主張「先預測再驗證」。請討論這個習慣為何能區分「工程」與「試誤」，並舉出本書中運用了同樣思維的其他例子。
3. 36.9.3 節主張誠實地寫出限制反而是優勢。請討論這與第 35.3.2 節「誠實的限制說明」的異同，並思考為何人們仍傾向隱藏缺點。
4. 36.5.1 節的垂直切片原則，與第 30.9 節的漸進式驗證、第 34.4.2 節的 POC 有共同的結構。請歸納這個結構，並說明它為何有效。
5. 回顧全書，哪一章的內容讓你最意外？哪一個觀念改變了你原本的想法？請具體說明。

章末摘要

- 專題的價值不在於嘗試了多少，而在於證明了什麼——關鍵是「你怎麼知道」與「別人能否相信」。
- 專題的四種價值：新穎性、性能、整合、理解；大學部專題最容易在後兩者創造價值。
- 一個被充分理解的失敗，比一個沒被理解的成功更有價值。
- 選題的三個判準：有價值、做得完、能學到東西；第二項最常被低估。
- 估算時間時應乘以 2.5，這符合「規劃謬誤」的普遍現象。

- 題目的三種來源：真實的痛點、延續既有工作、複現與延伸；複現本身就有價值。
- 選題健康檢查最重要的一問：若最難的部分失敗了，還剩下什麼？
- 不要讓任何單一的外部依賴（如未到貨的硬體）成為整個專題的必要條件。
- 界定範圍最有效的一招是寫下「不做什麼」，這能保護自己、聚焦資源、並誠實呈現。
- 每個專題都有「若失敗則全盤皆輸」的環節，必須提前識別並準備退路。
- 用倒推法規劃：扣除報告、測試與 30% 緩衝後，實際的開發時間常只有預期的一半。
- 里程碑必須可驗證；「端到端跑通一次」應該是很早的里程碑。
- 垂直切片優於水平堆疊：先做粗糙但完整的最小可行系統，在前三分之一完成。
- 版本管理不是選配；實驗紀錄有複利效應，應在實驗後立刻寫。
- 除錯的四原則：二分法、驗證假設而非猜測、一次只改一件事、能重現才能修。
- 量化的三要素：在什麼條件下、測了幾次、結果如何；缺一項結論就無法檢驗。
- 測試應涵蓋標準、變異、邊界、異常四類情境；邊界情境最能展現專業度。
- 每個情境至少 30 次；10 次全成功只能支持「大於 74%」的結論。
- 沒有基準線的性能數字沒有意義；消融實驗能驗證每個元件是否真有貢獻。
- 失敗分析應逐一分類並用五個為什麼追到根本原因——把「不知道怎麼辦」轉為「知道要改哪裡」。
- 改良的四步循環：提出方案、預測效果、實作重測、比對預測——先預測再驗證是工程與試誤的分界。
- 報告中「結果與討論」應佔一半篇幅；誠實地寫出限制反而顯示專業度。
- 簡報不是報告的縮短版；結果應佔最多時間，並用影片展示（含一段失敗的案例）。
- 練習三十秒的電梯簡報：問題、缺口、方法、驗證、貢獻五句話。

觀念題與實踐題

觀念題

1. 說明專題價值的三個核心問題，並解釋「嘗試」與「證明」的差別。
2. 列舉專題的四種價值來源，並說明大學部專題最容易在哪兩項創造價值。
3. 解釋為何「一個被充分理解的失敗，比一個沒被理解的成功更有價值」。

4. 列舉選題的三個判準，並說明「規劃謬誤」與 2.5 倍法則。
5. 比較題目的三種來源及其優缺點，並說明「複現本身就有價值」的理由。
6. 列舉選題健康檢查的六個問題，並說明哪一個最重要。
7. 說明「等硬體到就開始」為何是危險的題目類型，以及三種因應方式。
8. 說明「寫下不做什麼」的三個好處。
9. 解釋為何準備 Plan B 是專業而非沒信心。
10. 說明倒推法的五個步驟，並計算 16 週專題的實際開發時間。
11. 比較里程碑的兩種寫法，並說明「端到端跑通」為何應是很早的里程碑。
12. 說明關鍵路徑的意義與辨識方法，以及三個實務建議。
13. 比較兩種團隊分工模式，並說明推薦的混合作法。
14. 說明垂直切片優於水平堆疊的三個理由。
15. 說明最小可行系統的設計原則與深化的順序。
16. 列舉版本管理的三個理由與三個實用習慣。
17. 說明實驗紀錄應包含的四個要素及其複利效應。
18. 列舉除錯的四個原則，並說明「能重現才能修」的意義。
19. 說明技術債的三種標記及其區別。
20. 說明量化的三個要素，以及為何缺一項結論就無法檢驗。
21. 列舉測試情境的四個類別，並說明哪一類最能展現專業度。
22. 說明樣本數的判準，並解釋「10 次全成功」在統計上的意義。
23. 說明基準線的三種來源，以及消融實驗的目的。
24. 說明失敗分析的作法，以及五個為什麼如何把現象轉為可修改的原因。
25. 說明改良的四步循環，並解釋「先預測再驗證」為何是工程與試誤的分界。
26. 列舉報告的七個章節及其論證功能，並指出最常被寫得薄弱的一節。
27. 說明摘要的四個要素，並解釋為何應包含失敗的分析。
28. 說明誠實地寫出限制的三個理由。
29. 比較報告與簡報的目的差異，並說明簡報中哪一段應佔最多時間。

30. 說明展示影片的三個製作建議，以及為何應包含失敗的案例。

實踐題

- 選題評估：**列出三個你有興趣的專題題目，用 36.2.1 節的三個判準各自評分並說明理由。
接著對得分最高的做完整的健康檢查（36.2.3 節的六個問題），特別詳細回答第四個問題。
- 範圍與風險：**為你選定的題目撰寫完整的範圍文件（兩欄式，各至少五項），並識別三個關鍵風險與其退路（36.3.3 節）。每個退路都要說明它會損失什麼、保留什麼。
- 時程規劃：**用倒推法為你的專題排出時程，計算實際可用的開發週數。設定 5 個可驗證的里程碑，並畫出工作的相依關係圖標出關鍵路徑。若算出的開發時間不足，說明你會如何調整範圍。
- 測試方案設計：**為你的專題設計完整的測試方案，涵蓋四類情境（36.7.2 節）。列出每個情境的具體條件、測試次數、以及要記錄的指標。並說明你的基準線是什麼。
- 失敗分析的演練：**回想一個你過去做過的專題或作業中失敗的經驗。用五個為什麼追到根本原因，並提出一個針對根本原因的改良方案。接著預測：若當時做了這個改良，結果會如何改變？

動手做與專題延伸

本章實作：你的專題

本章的實作就是完成你自己的專題，詳見前面「實作」一節的五階段行動清單。

專題延伸

- 專題後的復盤：**專題結束後（無論成敗），花兩小時做一次完整的復盤。回答五個問題：
（一）哪些時間被浪費了？（二）哪個決定是最關鍵的？（三）如果重來一次，最先改變什麼？（四）我學到最重要的一件事是什麼？（五）這個專題的哪一部分值得延續？把答案寫下來——這份文件在你下一個專題、以及求職面試時都會極有價值。
- 把專題變成作品集：**整理成一份對外的版本，包含：一段 90 秒的影片、一頁的圖文說明（含量化的結果）、以及 GitHub 的程式碼與說明文件。求職時，一個能實際展示的作品，勝過十行履歷上的描述。
- 進階：延續與深化。**若你的專題有值得延續的部分，考慮：（一）投稿學生競賽或研討會——這會逼你把工作整理到可被審查的水準；（二）開源它——讓後續的人能站在你的基礎

上；（三）寫成技術部落格——把你踩過的坑寫下來，這對後人的價值可能超過專題本身。本書多次強調「一個被理解的失敗是有價值的貢獻」——你的踩坑紀錄，正是這樣的貢獻。

參考文獻與延伸閱讀

- Booth, W. C., Colomb, G. G. and Williams, J. M., The Craft of Research, University of Chicago Press (研究方法與論證結構的經典教材) 。
- Zobel, J., Writing for Computer Science, 3rd ed., Springer (技術寫作的實用指南，含實驗設計與結果呈現) 。
- Brooks, F. P., The Mythical Man-Month, Addison-Wesley (軟體專案管理的經典，含規劃謬誤的深刻討論) 。
- Ries, E., The Lean Startup, Crown Business (最小可行產品與快速迭代的思想來源) 。
- Reason, J., Human Error, Cambridge University Press (失效分析與根本原因的理論基礎) 。
- Ohno, T., Toyota Production System, Productivity Press (五個為什麼的原始出處) 。
- Kahneman, D., Thinking, Fast and Slow, Farrar, Straus and Giroux (規劃謬誤等認知偏誤的系統性討論) 。
- Chacon, S. and Straub, B., Pro Git (線上免費)：版本管理的完整教材。
- Wilson, G. et al., "Good Enough Practices in Scientific Computing," PLOS Computational Biology, 2017 (研究專案的實務建議，含資料與程式管理) 。
- Duke, A., Thinking in Bets, Portfolio (在不確定性下決策，含「先預測再驗證」的思維) 。
- Reynolds, G., Presentation Zen, New Riders (簡報設計的實用指南) 。
- 本書第 2 章的學習方法、第 30 章的漸進式驗證、第 34 章的系統工程：本章的許多原則在前面各章已有對應，建議一併回顧。